

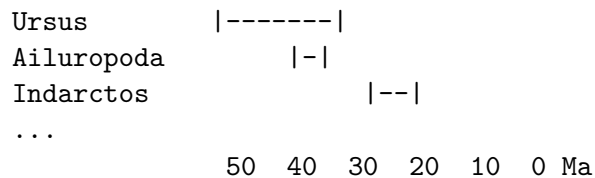
Range through time diversity analysis

1 Calculating ranges through time from occurrence data

1.1 Objectives

We are going to build a plot of the temporal ranges of bear genera (family Ursidae): one horizontal line per genus, spanning the time from its earliest to latest fossil occurrence. Time runs on the x-axis (reversed, so older is to the left), genus names on the y-axis.

By the end of this tutorial we will have produced something like this:



1.2 Environment setup

1.2.1 Load packages

We load the packages we need into the active environment with the `using` statements:

```
using DataFrames
using PaleobiologyDB
using PaleobiologyDB.Taxonomy
using Makie, CairoMakie
```

1.2.2 Optional: activate automatic data caching

If we want to store results locally to avoid querying the live database repeated, we can activate the PaleobiologyDB package's auto-caching mechanism:

```
PaleobiologyDB.set_autocaching!(true);
```

What does this do?

Activating caching stores the results of the PaleobiologyDB database queries locally so that repeated requests for the same data can be served instantly without re-downloading them. This improves performance and reduces network load, especially when working with large or frequently reused datasets. However, it can lead to outdated results if the underlying database changes, and it may consume disk space over time. You should use caching when you are repeatedly accessing the same data during exploratory analysis or development, but avoid it when you need the most current data or are working in contexts where storage or data freshness is critical.

1.3 Data acquisition

The initial raw dataset is obtained by downloading all fossil occurrence records of our study group using the `pbdb_occurrence` function of the PaleobiologyDB package:

```
using PaleobiologyDB

df_raw = pbdb_occurrences(
    base_name = "Ursidae";
```

```

show = "full",
vocab = "pbdb"
)

```

	occurrence_no	record_type	reid_no	flags	collection_no	identified_name	
	Int64	String3	Int64?	Missing	Int64	String	
1	138737	occ	<i>missing</i>	<i>missing</i>	11798	Indarctos sinensis	...
2	138739	occ	<i>missing</i>	<i>missing</i>	11798	Ursinae indet.	...
3	150068	occ	<i>missing</i>	<i>missing</i>	13293	Ursus thibetanus	...
4	150069	occ	<i>missing</i>	<i>missing</i>	13293	Ailuropoda melanoleuca	...
5	177833	occ	<i>missing</i>	<i>missing</i>	16885	Parictis parvus n. sp.	...
6	177834	occ	<i>missing</i>	<i>missing</i>	16885	Parictis gilpini n. sp.	...
7	177925	occ	3434	<i>missing</i>	16907	Parictis personi	...
8	178031	occ	<i>missing</i>	<i>missing</i>	16919	Parictis parvus	...
9	178233	occ	<i>missing</i>	<i>missing</i>	16933	Parictis gilpini	...
10	178370	occ	<i>missing</i>	<i>missing</i>	16946	Parictis montanus	...
11	178414	occ	<i>missing</i>	<i>missing</i>	16958	Parictis gilpini	...
12	178466	occ	<i>missing</i>	<i>missing</i>	16960	Subparictis parvus	...
13	178509	occ	10552	<i>missing</i>	16961	Subparictis parvus	...
14	178553	occ	<i>missing</i>	<i>missing</i>	16962	Parictis sp.	...
15	178604	occ	<i>missing</i>	<i>missing</i>	16963	Parictis sp.	...
16	178641	occ	<i>missing</i>	<i>missing</i>	16964	Parictis sp.	...
17	178837	occ	<i>missing</i>	<i>missing</i>	16988	Parictis montanus	...
18	178937	occ	<i>missing</i>	<i>missing</i>	16998	Parictis gilpini	...
19	178953	occ	<i>missing</i>	<i>missing</i>	17006	Parictis major n. sp.	...
20	179118	occ	<i>missing</i>	<i>missing</i>	17014	Parictis sp.	...
21	179189	occ	<i>missing</i>	<i>missing</i>	17024	Parictis gilpini	...
22	179293	occ	<i>missing</i>	<i>missing</i>	17040	Parictis gilpini	...
23	179327	occ	3805	<i>missing</i>	17045	Parictis parvus	...
24	179507	occ	<i>missing</i>	<i>missing</i>	17082	Drassonax harpagops n. sp.	...
...	...	...	...	...	...	...	...

1.4 Data cleaning

1.4.1 Structural cleaning: records with missing values in required fields

Our analysis requires records to have values in the taxonomic identity field (:accepted_name), as well as the chronological fields of interest (:early_interval, :max_ma, and :min_ma).

As a preliminary first-pass cleaning, we can ensure that these all records have values in these fields to avoid having to special-case handling `missing` data.

We use the `dropmissing` function from the `DataFrames` package to remove all records that are missing values in fields that we need.

```
using DataFrames

df = dropmissing(df_raw, [
    :genus,
    :early_interval,
    :max_ma,
    :min_ma
])
```

	occurrence_no	record_type	reid_no	flags	collection_no	identified_name	
	Int64	String3	Int64?	Missing	Int64	String	
1	138737	occ	<i>missing</i>	<i>missing</i>	11798	Indarctos sinensis	...
2	150068	occ	<i>missing</i>	<i>missing</i>	13293	Ursus thibetanus	...
3	150069	occ	<i>missing</i>	<i>missing</i>	13293	Ailuropoda melanoleuca	...
4	177833	occ	<i>missing</i>	<i>missing</i>	16885	Parictis parvus n. sp.	...
5	177834	occ	<i>missing</i>	<i>missing</i>	16885	Parictis gilpini n. sp.	...
6	177925	occ	3434	<i>missing</i>	16907	Parictis personi	...
7	178031	occ	<i>missing</i>	<i>missing</i>	16919	Parictis parvus	...
8	178233	occ	<i>missing</i>	<i>missing</i>	16933	Parictis gilpini	...
9	178370	occ	<i>missing</i>	<i>missing</i>	16946	Parictis montanus	...
10	178414	occ	<i>missing</i>	<i>missing</i>	16958	Parictis gilpini	...
11	178466	occ	<i>missing</i>	<i>missing</i>	16960	Subparictis parvus	...
12	178509	occ	10552	<i>missing</i>	16961	Subparictis parvus	...
13	178553	occ	<i>missing</i>	<i>missing</i>	16962	Parictis sp.	...
14	178604	occ	<i>missing</i>	<i>missing</i>	16963	Parictis sp.	...
15	178641	occ	<i>missing</i>	<i>missing</i>	16964	Parictis sp.	...
16	178837	occ	<i>missing</i>	<i>missing</i>	16988	Parictis montanus	...
17	178937	occ	<i>missing</i>	<i>missing</i>	16998	Parictis gilpini	...
18	178953	occ	<i>missing</i>	<i>missing</i>	17006	Parictis major n. sp.	...
19	179118	occ	<i>missing</i>	<i>missing</i>	17014	Parictis sp.	...
20	179189	occ	<i>missing</i>	<i>missing</i>	17024	Parictis gilpini	...
21	179293	occ	<i>missing</i>	<i>missing</i>	17040	Parictis gilpini	...
22	179327	occ	3805	<i>missing</i>	17045	Parictis parvus	...
23	179507	occ	<i>missing</i>	<i>missing</i>	17082	Drassonax harpagops n. sp.	...
24	179963	occ	<i>missing</i>	<i>missing</i>	17147	Parictis gilpini	...
...	...	...	...	...	...	...	...

1.4.2 Domain-scope cleaning

1.4.2.1 Removing records with insufficient taxonomic resolution

Our analysis requires that our fossil occurrence be identified *at least* to genus level. `## Computing midpoint ages ##`
`Computing midpoint ages ## Computing midpoint ages` This means filtering the dataset to retain records that have their `:accepted_rank` field values of “genus” (the identification of the fossil was to the level of genus), but also, addition, “species” and “subspecies” as well, as this level of identification qualifies. We will use the `Taxonomy.drop_unresolved_taxa` function, a

function in the `Taxonomy` submodule of the `PaleobiologyDB` module, to wrap up all this complexity to ensure resolution to the level of genus or better, passing in the string argument `"genus"` to indicate the taxonomic rank (we would use `"species"` if we wanted identification at species rank, `"family"` for family, and so on).

```
using PaleobiologyDB.Taxonomy: drop_unresolved_taxa
df = drop_unresolved_taxa(df, "genus")
```

	occurrence_no	record_type	reid_no	flags	collection_no	identified_name	
	Int64	String3	Int64?	Missing	Int64	String	
1	138737	occ	<i>missing</i>	<i>missing</i>	11798	Indarctos sinensis	...
2	150068	occ	<i>missing</i>	<i>missing</i>	13293	Ursus thibetanus	...
3	150069	occ	<i>missing</i>	<i>missing</i>	13293	Ailuropoda melanoleuca	...
4	177833	occ	<i>missing</i>	<i>missing</i>	16885	Parictis parvus n. sp.	...
5	177834	occ	<i>missing</i>	<i>missing</i>	16885	Parictis gilpini n. sp.	...
6	177925	occ	3434	<i>missing</i>	16907	Parictis personi	...
7	178031	occ	<i>missing</i>	<i>missing</i>	16919	Parictis parvus	...
8	178233	occ	<i>missing</i>	<i>missing</i>	16933	Parictis gilpini	...
9	178370	occ	<i>missing</i>	<i>missing</i>	16946	Parictis montanus	...
10	178414	occ	<i>missing</i>	<i>missing</i>	16958	Parictis gilpini	...
11	178466	occ	<i>missing</i>	<i>missing</i>	16960	Subparictis parvus	...
12	178509	occ	10552	<i>missing</i>	16961	Subparictis parvus	...
13	178553	occ	<i>missing</i>	<i>missing</i>	16962	Parictis sp.	...
14	178604	occ	<i>missing</i>	<i>missing</i>	16963	Parictis sp.	...
15	178641	occ	<i>missing</i>	<i>missing</i>	16964	Parictis sp.	...
16	178837	occ	<i>missing</i>	<i>missing</i>	16988	Parictis montanus	...
17	178937	occ	<i>missing</i>	<i>missing</i>	16998	Parictis gilpini	...
18	178953	occ	<i>missing</i>	<i>missing</i>	17006	Parictis major n. sp.	...
19	179118	occ	<i>missing</i>	<i>missing</i>	17014	Parictis sp.	...
20	179189	occ	<i>missing</i>	<i>missing</i>	17024	Parictis gilpini	...
21	179293	occ	<i>missing</i>	<i>missing</i>	17040	Parictis gilpini	...
22	179327	occ	3805	<i>missing</i>	17045	Parictis parvus	...
23	179507	occ	<i>missing</i>	<i>missing</i>	17082	Drassonax harpagops n. sp.	...
24	179963	occ	<i>missing</i>	<i>missing</i>	17147	Parictis gilpini	...
...	...	...	...	...	...	...	...

1.4.2.2 Removing records with inaccurate taxonomic resolution

Our dataset now has records in which the fossil has been identified to at least genus level. However, we also want to be sure that the taxon name we are going to use to assign the fossil to its analytical unit is actually a registered taxon name in the PBDB database, to avoid creating groups based on typos, misspelling, or other similar domain-side (rather than structural) data errors. We use `Taxonomy.drop_unrecognized_taxa(df, :genus)` to ensure the names given in the `:genus` field of the dataframe is found the PBDB taxonomy database. (If we wanted a lower resolution in our taxonomy, e.g. at the family level, we would use `:family`, or for higher resolution, i.e. species level, we would use `:accepted_name`, as the PBDB data does not track the full species epithet in a separate field).

```
using PaleobiologyDB.Taxonomy: drop_unrecognized_taxa
df = Taxonomy.drop_unrecognized_taxa(df, :genus)
```

	occurrence_no	record_type	reid_no	flags	collection_no	identified_name	
	Int64	String3	Int64?	Missing	Int64	String	
1	138737	occ	<i>missing</i>	<i>missing</i>	11798	Indarctos sinensis	...
2	150068	occ	<i>missing</i>	<i>missing</i>	13293	Ursus thibetanus	...
3	150069	occ	<i>missing</i>	<i>missing</i>	13293	Ailuropoda melanoleuca	...
4	177833	occ	<i>missing</i>	<i>missing</i>	16885	Parictis parvus n. sp.	...
5	177834	occ	<i>missing</i>	<i>missing</i>	16885	Parictis gilpini n. sp.	...
6	177925	occ	3434	<i>missing</i>	16907	Parictis personi	...
7	178031	occ	<i>missing</i>	<i>missing</i>	16919	Parictis parvus	...
8	178233	occ	<i>missing</i>	<i>missing</i>	16933	Parictis gilpini	...
9	178370	occ	<i>missing</i>	<i>missing</i>	16946	Parictis montanus	...
10	178414	occ	<i>missing</i>	<i>missing</i>	16958	Parictis gilpini	...
11	178466	occ	<i>missing</i>	<i>missing</i>	16960	Subparictis parvus	...
12	178509	occ	10552	<i>missing</i>	16961	Subparictis parvus	...
13	178553	occ	<i>missing</i>	<i>missing</i>	16962	Parictis sp.	...
14	178604	occ	<i>missing</i>	<i>missing</i>	16963	Parictis sp.	...
15	178641	occ	<i>missing</i>	<i>missing</i>	16964	Parictis sp.	...
16	178837	occ	<i>missing</i>	<i>missing</i>	16988	Parictis montanus	...
17	178937	occ	<i>missing</i>	<i>missing</i>	16998	Parictis gilpini	...
18	178953	occ	<i>missing</i>	<i>missing</i>	17006	Parictis major n. sp.	...
19	179118	occ	<i>missing</i>	<i>missing</i>	17014	Parictis sp.	...
20	179189	occ	<i>missing</i>	<i>missing</i>	17024	Parictis gilpini	...
21	179293	occ	<i>missing</i>	<i>missing</i>	17040	Parictis gilpini	...
22	179327	occ	3805	<i>missing</i>	17045	Parictis parvus	...
23	179507	occ	<i>missing</i>	<i>missing</i>	17082	Drassonax harpagops n. sp.	...
24	179963	occ	<i>missing</i>	<i>missing</i>	17147	Parictis gilpini	...
...	...	...	...	...	...	...	...

1.4.2.3 Dropping records with *either* imprecise or inaccurate taxonomy

In fact, *both* operations can be combined using the `drop_unqualified_taxa` function:

```
#| output-fold: true
using PaleobiologyDB.Taxonomy: drop_unqualified_taxa
df = Taxonomy.drop_unrecognized_taxa(df, "genus")
```


1.5 Computing a midpoint age

Each fossil occurrence in the PBDB has an age *range*: `max_ma` (the older end) and `min_ma` (the more recent end), both in millions of years. For our chart we want a single number to represent each occurrence’s age. We will use the midpoint: $\frac{\text{max_ma} + \text{min_ma}}{2}$.

Adding a new column to a DataFrame is straightforward — assign to a column name that does not yet exist:

```
df.new_column = ...
```

That creates the column in place.

Vectorized (dot) operators — the `.+` and `./` in our expression — mean “apply this operation element by element across the whole column”. Without the dot, Julia would try to operate on the column as a single object, which is not what we want here.

```
df.midpoint_age = (df.max_ma .+ df.min_ma) ./ 2
```

```
979-element Vector{Float64}:
```

```
 9.438
 0.39285000000000003
 0.39285000000000003
35.45
35.45
35.45
35.45
35.45
35.45
35.45

 0.00585
 0.07035
 1.677
 1.677
 1.29
 1.29
```

```
0.00585
0.00585
0.4515
```

1.6 Sorting

`sort(df, cols)` returns a new DataFrame with rows ordered by the given columns. It does not modify the original — we reassign the result back to `df`.

```
df = sort(df, [:genus, :midpoint_age])
```

	occurrence_no	record_type	reid_no	flags	collection_no	identified_name	
	Int64	String3	Int64?	Missing	Int64	String	
1	1353531	occ	<i>missing</i>	<i>missing</i>	36712	Adelpharctos ginsburgi n. sp.	...
2	1587709	occ	<i>missing</i>	<i>missing</i>	182805	Adelpharctos mirus	...
3	565656	occ	<i>missing</i>	<i>missing</i>	36709	Adelpharctos mirus n. gen. n. sp.	...
4	1635457	occ	<i>missing</i>	<i>missing</i>	231920	Agriarctos nikolovi	...
5	1635458	occ	<i>missing</i>	<i>missing</i>	231920	Agriarctos nikolovi	...
6	838805	occ	<i>missing</i>	<i>missing</i>	93106	Agriotherium cf. schneideri	...
7	1519354	occ	<i>missing</i>	<i>missing</i>	20079	Agriotherium schneideri	...
8	978925	occ	<i>missing</i>	<i>missing</i>	21713	Agriotherium africanum	...
9	1413754	occ	<i>missing</i>	<i>missing</i>	185859	Agriotherium sp.	...
10	501831	occ	<i>missing</i>	<i>missing</i>	50943	Agriotherium sp.	...
11	435589	occ	<i>missing</i>	<i>missing</i>	33396	Agriotherium intermedium	...
12	432641	occ	<i>missing</i>	<i>missing</i>	42273	Agriotherium sp.	...
13	227614	occ	<i>missing</i>	<i>missing</i>	22221	Agriotherium africanum	...
14	227627	occ	<i>missing</i>	<i>missing</i>	22222	Agriotherium africanum	...
15	659279	occ	<i>missing</i>	<i>missing</i>	22160	Agriotherium sp.	...
16	908635	occ	<i>missing</i>	<i>missing</i>	102372	? Agriotherium sp.	...
17	1357547	occ	<i>missing</i>	<i>missing</i>	184410	Agriotherium cf. sivalensis	...
18	741192	occ	<i>missing</i>	<i>missing</i>	59167	Agriotherium africanum	...
19	908633	occ	<i>missing</i>	<i>missing</i>	102370	Agriotherium sp.	...
20	419634	occ	<i>missing</i>	<i>missing</i>	40758	Agriotherium roblesi n. sp.	...
21	184173	occ	<i>missing</i>	<i>missing</i>	18036	Agriotherium sp.	...
22	184701	occ	5518	<i>missing</i>	18097	Agriotherium schneideri	...
23	184926	occ	<i>missing</i>	<i>missing</i>	18126	Agriotherium sp.	...
24	185007	occ	<i>missing</i>	<i>missing</i>	18140	Agriotherium sp.	...
...	...	...	...	...	...	...	...

This puts all rows of the same genus together, with older occurrences first within each genus.

1.7 Split-Apply-Combine

Key concepts: Split-Apply-Combine

We need one row per genus with its earliest and latest appearance. The fossil data has *many* rows per genus. The strategy: *split* the big table into per-genus sub-tables, *apply* a summary function to each, then *combine* results into a new table.

We run this algorithm elegantly in a single step using the `combine` function in conjunction with the `group` function.

```
appearance_df = combine(  
  groupby(df, :genus; sort = true),  
  :midpoint_age => maximum => :first_appearance_datum,  
  :midpoint_age => minimum => :last_appearance_datum  
)
```

The `combine` function takes a `GroupedDataFrame` as its first argument. The first argument passed to the `combine` function here is the result of:

```
groupby(df, :genus)`
```

This function splits `df` into groups on the `:genus` column and returns `GroupedDataFrame`.

The remaining arguments specify *what* data in the group is to be summarized, *how* that data in the groups are to be summarized, and the *name* to call the summary.

So,

```
:midpoint_age => maximum => :first_appearance_datum,
```

means: in each group, take the `:midpoint_age` column, apply `maximum` to it, store the result in column `:first_appearance_datum`.

while

```
:midpoint_age => minimum => :last_appearance_datum,
```

means: in each group, take the `:midpoint_age` column, apply `minimum` to it, store the result in column `:last_appearance_datum`.

Right now `df` has one row per fossil *occurrence* — potentially

many rows per genus. We need a summary table with one row per *genus*, recording the earliest and latest midpoint age for that genus.

The strategy for collapsing many rows into a summary is called **Split-Apply-Combine**:

1. **Split** the big table into sub-tables, one per genus.
2. **Apply** a summary function to each sub-table.
3. **Combine** the results into a new table.

1.7.1 Step 1 — Split: `groupby`

`groupby(df, :genus)` partitions the DataFrame into a collection of sub-DataFrames, one for each distinct genus value.

```
gdf = groupby(df, :genus)
```

Each element of `gdf` is a small DataFrame containing only the rows for one genus — you can think of it as a pile of index cards sorted by genus name.

1.7.2 Step 2 — Apply a function to a column: the `=>` syntax

`combine` uses a compact notation to describe what to compute for each group:

```
:source_column => function => :result_column
```

For example:

```
:midpoint_age => maximum => :first_appearance_datum
```

means: in each group, take the `:midpoint_age` column, apply `maximum` to it (giving the oldest age, i.e. the first appearance), and store that result in a new column called `:first_appearance_datum`.

i Why maximum = first appearance?

Ages are in millions of years *ago*, so a *larger* number means *further back in time*. `maximum(:midpoint_age)` gives the largest value, which is the oldest occurrence — the first appearance. `minimum(:midpoint_age)` gives the most recent — the last appearance.

1.7.3 Step 3 — Combine: the full expression

```
appearance_df = combine(  
  groupby(df, :genus; sort = true),  
  :midpoint_age => maximum => :first_appearance_datum,  
  :midpoint_age => minimum => :last_appearance_datum  
)
```

	genus	first_appearance_datum	last_appearance_datum
	String31	Float64	Float64
1	Adelpharctos	30.6	25.17
2	Agriarctos	8.4815	8.4815
3	Agriotherium	14.1865	3.05
4	Ailuropoda	3.09	0.07035
5	Amphicynodon	30.6	25.17
6	Arctodus	3.05	0.0645
7	Arctotherium	3.05	0.0645
8	Arctotherium (Arctotherium)	1.677	1.677
9	Arctotherium (Pararctotherium)	1.29	0.0645
10	Ballusia	18.45	13.848
11	Cephalogale	30.6	17.4
12	Cyonarctos	25.17	25.17
13	Dinocyon	17.335	14.1865
14	Drassonax	32.85	32.85
15	Hemicyon	18.45	10.95
16	Indarctos	14.4	4.4665
17	Kretzoiarctos	10.1775	9.438
18	Melursus	1.29585	1.29585
19	Metarctos	8.4815	8.4815
20	Miomaci	10.385	10.385
21	Pachycynodon	30.6	21.745
22	Parictis	35.45	32.85
23	Phoberocyon	25.17	13.848
24	Phoberogale	30.6	21.2
...	...	...	...

`combine` runs both transformations on every group and assembles the results into a new `DataFrame` — one row per genus, with three columns: `:genus`, `:first_appearance_datum`, and `:last_appearance_datum`.

1.7.4 Sort the summary table

We want genera ordered by their first appearance (oldest first) for the chart. `rev=true` sorts in descending order (largest value first):


```
appearance_df = sort(appearance_df, :first_appearance_datum, rev=true)
```

	genus	first_appearance_datum	last_appearance_datum
	String31	Float64	Float64
1	Parictis	35.45	32.85
2	Drassonax	32.85	32.85
3	Adelpharctos	30.6	25.17
4	Amphicynodon	30.6	25.17
5	Cephalogale	30.6	17.4
6	Pachycynodon	30.6	21.745
7	Phoberogale	30.6	21.2
8	Proplesictis	30.6	30.6
9	Wangictis	30.6	30.6
10	Cyonarctos	25.17	25.17
11	Phoberocyon	25.17	13.848
12	Ursavus	24.0	6.9075
13	Zaragocyon	21.2	21.2
14	Ballusia	18.45	13.848
15	Hemicyon	18.45	10.95
16	Plithocyon	18.215	10.6565
17	Dinocyon	17.335	14.1865
18	Indarctos	14.4	4.4665
19	Agriotherium	14.1865	3.05
20	Ursus	12.81	0.00585
21	Miomaci	10.385	10.385
22	Kretzoiarctos	10.1775	9.438
23	Agriarctos	8.4815	8.4815
24	Metarctos	8.4815	8.4815
...	...	...	...

1.8 Building the chart

1.8.1 Setting up the figure and axes

You already know the Figure / Axis pattern. Two new axis options appear here:

- **yticks** takes a pair (**positions**, **labels**) — numeric positions on the y-axis and the text labels to show at those positions. We use `1:nrow(appearance_df)` as positions (one integer per genus) and `appearance_df.genus` as the labels.
- **xreversed = true** flips the x-axis so that older ages (larger numbers) are on the left, matching the geological convention.

```
using Makie, CairoMakie

fig = Figure()
ax = Axis(
    fig[1, 1],
    xlabel = "Midpoint age (Ma)",
    ylabel = "Genus",
    yticks = (1:nrow(appearance_df), appearance_df.genus),
    xreversed = true
)
```

Makie.Axis with 0 plots:

1.8.2 Drawing the lines with a loop

We need one horizontal line per genus. We *could* write out `lines!()` once for each genus by hand — but that would be tedious, error-prone, and would break the moment the number of genera changed.

Instead we use a **for loop**: write the instruction once, and tell Julia to repeat it for each genus automatically.

The basic structure is:

```
for i in 1:5
    println(i)
end
```

`1:5` is a *range* — the values 1, 2, 3, 4, 5. On each pass through the loop, `i` takes the next value from the range. Everything between **for** and **end** runs once per value.

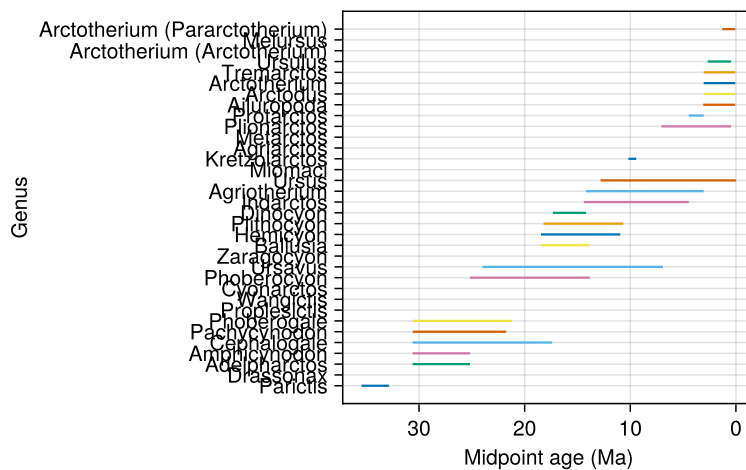
In our case the range is `1:nrow(appearance_df)` — one integer for each row (i.e. each genus) in our summary table. We use `i` for two purposes simultaneously:

- As a **row index** to look up that genus's dates: `appearance_df.first_appearance_datum[i]`
- As the **y-position** of the line on the chart (genus 1 at height 1, genus 2 at height 2, ...).

`lines!(ax, [x1, x2], [i, i])` draws a horizontal segment: the x-coordinates go from `x1` to `x2`, and both y-coordinates are `i`, so the line is flat at that height.

```
for i in 1:nrow(appearance_df)
  x1 = appearance_df.first_appearance_datum[i]
  x2 = appearance_df.last_appearance_datum[i]
  lines!(ax, [x1, x2], [i, i])
end

fig
```



1.9 Complete program

For reference, here is the full program in one place:

```
using DataFrames
using PaleobiologyDB
using PaleobiologyDB.Taxonomy
using Makie, CairoMakie

PaleobiologyDB.set_autocaching!(true);

using PaleobiologyDB

df_raw = pbdb_occurrences(
    base_name = "Ursidae";
    show = "full",
    vocab = "pbdb"
)

using DataFrames

df = dropmissing(df_raw, [
    :genus,
    :early_interval,
    :max_ma,
    :min_ma
])

using PaleobiologyDB.Taxonomy: drop_unresolved_taxa
df = drop_unresolved_taxa(df, "genus")

using PaleobiologyDB.Taxonomy: drop_unrecognized_taxa
df = Taxonomy.drop_unrecognized_taxa(df, :genus)

df.midpoint_age = (df.max_ma .+ df.min_ma) ./ 2

df = sort(df, [:genus, :midpoint_age])

appearance_df = combine(
    groupby(df, :genus; sort = true),
    :midpoint_age => maximum => :first_appearance_datum,
```

```

        :midpoint_age => minimum => :last_appearance_datum
    )

    appearance_df = sort(appearance_df, :first_appearance_datum, rev=true)

    using Makie, CairoMakie

    fig = Figure()
    ax = Axis(
        fig[1, 1],
        xlabel = "Midpoint age (Ma)",
        ylabel = "Genus",
        yticks = (1:nrow(appearance_df), appearance_df.genus),
        xreversed = true
    )

    for i in 1:nrow(appearance_df)
        x1 = appearance_df.first_appearance_datum[i]
        x2 = appearance_df.last_appearance_datum[i]
        lines!(ax, [x1, x2], [i, i])
    end

    fig

```

1.10 Exercises: Calculating ranges through time

1.10.1 Julia concepts and mechanisms

For each snippet, read the code and predict what it will print *before* you run it. Then try it in the REPL to check.

1.10.1.1 Part A: Vectorised operations

1. Dividing every element:

```

a = [10.0, 20.0, 30.0]
println(a ./ 10.0)

```

i Answer

[1.0, 2.0, 3.0]
./ 10.0 divides each element of **a** by 10 independently.

2. Adding two arrays element-by-element:

```
max_ma = [40.0, 20.0]
min_ma = [30.0, 10.0]
println(max_ma .+ min_ma)
```

i Answer

[70.0, 30.0]

3. Computing midpoints:

```
max_ma = [55.0, 33.9, 23.0]
min_ma = [33.9, 23.0, 5.3]
midpoints = (max_ma .+ min_ma) ./ 2
println(midpoints)
```

i Answer

[44.45, 28.45, 14.15]
.+ adds corresponding elements; ./ 2 then divides each result.

4. Dot multiplication:

```
ages = [10.0, 20.0, 30.0]
println(ages .* 2.0)
```

i Answer

[20.0, 40.0, 60.0]

5. Subtracting a scalar from every element:

```
ages = [50.0, 30.0, 10.0]
println(ages .- 5.0)
```

i Answer

[45.0, 25.0, 5.0]

6. Chaining dot operations:

```
a = [8.0, 16.0, 24.0]
b = [2.0, 4.0, 6.0]
println((a .+ b) ./ 2.0)
```

i Answer

[5.0, 10.0, 15.0]

The parentheses force .+ first; ./ 2.0 then halves each sum.

1.10.1.2 Part B: Loops and indexing

1. A basic for loop:

```
for i in 1:4
    println(i * 10)
end
```

i Answer

10

20

30

40

i takes each value in the range 1:4 in turn.

2. Using i as both an index and a display number:

```
genera = ["Ursus", "Ailuropoda", "Indarctos"]
for i in 1:3
    println(i, ". ", genera[i])
end
```

i Answer

1. Ursus
2. Ailuropoda
3. Indarctos

This is the same dual role `i` plays in the range chart:
row index *and* y-position.

3. Squaring inside a loop:

```
for i in 1:5
    println(i ^ 2)
end
```

i Answer

1
4
9
16
25

4. Accessing the last element with `end`:

```
v = [10, 20, 30, 40, 50]
println(v[end])
```

i Answer

50
`end` always refers to the last valid index, regardless
of array length.

5. Range length:

```
r = 1:6
println(length(r))
```

i Answer

6

6. Loop over an array using `length`:


```
fads = [45.0, 30.0, 20.0]
for i in 1:length(fads)
    println("Genus ", i, " appeared at ", fads[i], " Ma")
end
```

i Answer

```
Genus 1 appeared at 45.0 Ma
Genus 2 appeared at 30.0 Ma
Genus 3 appeared at 20.0 Ma
```

7. Adding the loop variable to the element:

```
v = [10, 20, 30]
for i in 1:length(v)
    println(v[i] + i)
end
```

i Answer

```
11
22
33
v[1]+1=11, v[2]+2=22, v[3]+3=33.
```

8. First and last elements after sorting:

```
data = [5.0, 3.0, 8.0, 1.0, 9.0]
sorted = sort(data)
println(sorted[1])
println(sorted[end])
```

i Answer

```
1.0
9.0

sort returns a new array in ascending order. [1] is
the smallest, [end] the largest.
```

1.10.1.3 Part C: Sorting and aggregation

1. Default sort (ascending):

```
v = [30.0, 10.0, 50.0, 20.0, 40.0]
println(sort(v))
```

i Answer

```
[10.0, 20.0, 30.0, 40.0, 50.0]
```

2. Sort descending:

```
v = [30.0, 10.0, 50.0, 20.0, 40.0]
println(sort(v, rev=true))
```

i Answer

```
[50.0, 40.0, 30.0, 20.0, 10.0]
```

`rev=true` reverses the default ordering.

3. Finding FAD and LAD from a vector of midpoint ages:

```
midpoint_ages = [5.0, 45.0, 23.0, 12.0, 8.0]
fad = maximum(midpoint_ages)
lad = minimum(midpoint_ages)
println("FAD: ", fad, " Ma")
println("LAD: ", lad, " Ma")
```

i Answer

FAD: 45.0 Ma

LAD: 5.0 Ma

Geological ages are measured in Ma (millions of years ago), so the largest value is the oldest — the first appearance.

4. `maximum` and `minimum` on a simple array:

```
ages = [3.0, 7.0, 1.5, 12.0, 4.0]
println(maximum(ages))
println(minimum(ages))
```

i Answer

12.0
1.5

5. Finding which genus appeared first using `argmax`:

```
genera = ["A", "B", "C"]  
fads    = [10.0, 45.0, 28.0]  
idx = argmax(fads)  
println(genera[idx], " appeared first")
```

i Answer

B appeared first
`argmax(fads)` returns 2 (the index of 45.0, the largest value); `genera[2]` is "B".

6. Sorting a DataFrame by a column:

```
using DataFrames  
df = DataFrame(genus = ["C", "A", "B"], fad = [10.0, 50.0, 30.0])  
result = sort(df, :fad, rev=true)  
println(result.genus)
```

i Answer

["A", "B", "C"]
`rev=true` puts the largest FAD first. "A" has FAD 50.0, "B" 30.0, "C" 10.0.

1.10.1.4 Part D: DataFrames and split-apply-combine

These fragments require `using DataFrames`.

1. How many rows after combine?

```
using DataFrames  
df = DataFrame(  
    genus = ["X", "X", "Y", "Y", "Y"],  
    age    = [20.0, 40.0, 5.0, 15.0, 10.0]  
)
```

```
result = combine(
  groupby(df, :genus),
  :age => maximum => :fad,
  :age => minimum => :lad,
)
println(nrow(result))
```

i Answer

2

combine produces one output row per group. Two distinct genera → two rows.

2. Reading the FAD values out of the result:

```
using DataFrames
df = DataFrame(
  genus = ["A", "A", "B", "B"],
  age    = [10.0, 30.0, 5.0, 15.0]
)
result = combine(groupby(df, :genus), :age => maximum => :fad)
println(result.fad)
```

i Answer

[30.0, 15.0]

Genus A: maximum([10.0, 30.0]) = 30.0. Genus

B: maximum([5.0, 15.0]) = 15.0.

3. Adding a new column to a DataFrame:

```
using DataFrames
df = DataFrame(max_ma = [40.0, 20.0, 10.0], min_ma = [30.0, 10.0, 5.0])
df.midpoint = (df.max_ma .+ df.min_ma) ./ 2
println(df.midpoint)
```

i Answer

[35.0, 15.0, 7.5]

Assigning to df.midpoint creates the column in

place.

4. `nrow` on a `DataFrame`:

```
using DataFrames
df = DataFrame(genus = ["A","A","B","C","C","C"], age = [1,2,3,4,5,6])
println(nrow(df))
```

i Answer

6

`nrow` counts all rows, not groups.

5. Finding the genus with the oldest FAD:

```
using DataFrames
df = DataFrame(
    genus = ["Ursus", "Ursus", "Ailuropoda", "Ailuropoda"],
    age    = [0.5, 1.2, 3.0, 6.0]
)
result = combine(groupby(df, :genus), :age => maximum => :fad)
oldest = result.genus[argmax(result.fad)]
println(oldest)
```

i Answer

Ailuropoda

The maximum FAD is 6.0, which belongs to Ailuropoda.

6. Sorting after `combine`:

```
using DataFrames
df = DataFrame(
    genus = ["C","C","A","A","B"],
    age    = [5.0, 8.0, 20.0, 35.0, 1.0]
)
result = combine(groupby(df, :genus), :age => maximum => :fad)
result = sort(result, :fad, rev=true)
println(result.genus)
```

i Answer

["A", "C", "B"]

FADs: $A \rightarrow 35$, $C \rightarrow 8$, $B \rightarrow 1$. Sorted largest-first: A, C, B.

7. What do the three parts of the `=>` chain mean?

```
:midpoint_age => maximum => :first_appearance_datum
```

No running needed — identify each part.

i Answer

1. `:midpoint_age` — the **source column** within each group
2. `maximum` — the **function** applied to that column
3. `:first_appearance_datum` — the **name** given to the result column

Read it as: “take `:midpoint_age`, apply `maximum`, store the result as `:first_appearance_datum`.”

8. `sort` returns a new DataFrame; column assignment modifies in place:

```
using DataFrames
df = DataFrame(x = [3, 1, 2])
df.y = df.x .* 10
println(df.y)
sort(df, :x)          # does this change df?
println(df.x)
```

i Answer

[30, 10, 20]

[3, 1, 2]

`df.y = ...` modifies `df` in place. `sort(df, :x)` returns a *new* sorted DataFrame but does not change `df` — you would need `df = sort(df, :x)` to reas-

sign.

9. Computing range duration with dot subtraction:

```
using DataFrames
df = DataFrame(
  genus = ["A", "B", "C"],
  fad    = [10.0, 50.0, 30.0],
  lad    = [2.0,  5.0,  1.0],
)
df.duration = df.fad .- df.lad
println(df.duration)
```

i Answer

[8.0, 45.0, 29.0]

Duration = FAD – LAD, computed element-by-element.

10. Keyword argument `rev = false` is the default:

```
result = sort([5, 3, 8, 1], rev = false)
println(result)
```

i Answer

[1, 3, 5, 8]

`rev = false` is ascending order — the same as calling `sort([5, 3, 8, 1])` with no keyword.

1.10.2 Practical shorts

Assume you have completed the tutorial and have `df`, `appearance_df`, and the range chart available in your REPL.

1. How many genera are in `appearance_df`? Print the count with `nrow(appearance_df)`.

2. Print just the first five rows of `appearance_df`:
`appearance_df[1:5, :]`
3. Compute the fossil record duration for each genus. Add a column `duration` to `appearance_df` where `duration = first_appearance_datum - last_appearance_datum`.
4. Which genus has the longest fossil record? Use `argmax(appearance_df.duration)` to get the row index, then print `appearance_df.genus[idx]`.
5. Which genus has the shortest fossil record? Use `argmin`.
6. How many genera have a `first_appearance_datum` older than 20 Ma? Try: `count(x -> x > 20, appearance_df.first_appearance_datum)`. (We cover `count` formally in the next section — this is a preview.)
7. Repeat the full pipeline for cats instead of bears. Change `base_name = "Ursidae"` to `base_name = "Felidae"`. How many genera does Felidae produce?
8. Sort `appearance_df` by `last_appearance_datum` in ascending order (`rev = false`). Which genus disappeared first (has the highest LAD in Ma)?
9. Swap `rev=true` for `rev=false` in the final sort of `appearance_df` and redraw the chart. How does the order of genera on the y-axis change?
10. Compare `nrow(df_raw)` with `nrow(df)` after all cleaning. How many records were removed in total?
11. Print all genus names from `appearance_df.genus`.
12. Compute the mean first-appearance datum: `sum(appearance_df.first_appearance_datum) / nrow(appearance_df)`. What does this number represent biologically?
13. Add a title to the chart. Pass `title = "Ursidae genera through time"` to the `Axis(...)` call and redraw.
14. Change the line color in the range chart. Add `color = :steelblue` to the `lines!(ax, ...)` call inside the loop and redraw.

15. Draw only genera with a range longer than 10 Ma. Add an if check inside the loop:

```
if x2 - x1 > 10.0
  lines!(ax, [x1, x2], [i, i])
end
```

16. Download occurrence data for Rhinocerotidae (rhinos) and run the full pipeline. How does the range chart compare visually to the bear chart?
17. Find the `first_appearance_datum` of the genus *Ursus*. Filter: `appearance_df[appearance_df.genus .== "Ursus", :]`
18. Remove `sort = true` from `groupby(df, :genus; sort = true)`. Does the order of rows in `appearance_df` change? Does the chart change?
19. Make the lines thicker. Add `linewidth = 2` to the `lines!()` call inside the loop.
20. How many occurrences in the cleaned `df` belong to the genus *Ursus*? `nrow(df[df.genus .== "Ursus", :])`
21. Compare `dropmissing` applied only to `:genus` versus all four required columns. How many rows differ between the two results?
22. What is the largest `max_ma` value in the raw dataset? `maximum(skipmissing(df_raw.max_ma))`
23. Print the three oldest genera (by FAD). After sorting by FAD descending, use: `appearance_df[1:3, [:genus, :first_appearance_datum]]`
24. Recompute midpoint age using only `max_ma` (no averaging). Replace `df.midpoint_age = (df.max_ma .+ df.min_ma) ./ 2` with `df.midpoint_age = df.max_ma`. Does the range chart change noticeably?
25. Set `xreversed = false` in the `Axis` call and redraw. What is visually wrong with the time axis now?
26. Update the axis labels. Set `xlabel = "Millions of years ago"` and `ylabel = "Bear genus"`.

27. Which genus both appeared earliest *and* disappeared most recently? That is: which genus has the largest `first_appearance_datum` *and* the smallest `last_appearance_datum`? (There is likely no single genus satisfying both perfectly — so rank them and compare.)
 28. Create a DataFrame `likely_extant` containing only genera whose `last_appearance_datum` is less than 1.0 Ma.
 29. Print the genus names in `appearance_df` sorted alphabetically. `sort(appearance_df.genus)`
 30. Reason through: how would the number of genera in `appearance_df` change if you added `:family` to the `dropmissing` call? Then test it.
-

1.10.3 Creative programming

1.10.3.1 Range chart for a taxon of your choice

Pick any mammal family from the Paleobiology Database — for example Equidae (horses), Cervidae (deer), or Proboscidea (elephants and mammoths) — and build a complete range-through chart from scratch following the same pipeline as this tutorial.

Objectives

- Download and clean occurrence data for your chosen taxon.
- Compute FAD and LAD per genus and create the range chart.
- Add a descriptive title and clear axis labels.
- Compare the chart visually with the Ursidae chart: which group has more genera? a longer total record?

Hints

- Start by changing only `base_name`; adjust cleaning steps if the new taxon behaves differently.

- For taxa with many genera (e.g. Dinosauria), the figure may need a larger size: `Figure(size = (800, 1200))`.

1.10.3.2 Color-coded survivors and extinct genera

Modify the Ursidae range chart so that genera likely still alive today are drawn in one color and genera that went extinct before the Holocene are drawn in another.

Objectives

- Define “likely extant” as `last_appearance_datum < 1.0` Ma.
- Draw extant genera with `color = :steelblue` and extinct genera with `color = :salmon`.
- Add a legend explaining the color coding.

Hints

- Use an `if/else` block inside the loop to choose the color before calling `lines!()`.
- Browse the Makie documentation for `axislegend` or `Legend` to add a legend to the figure.

1.10.3.3 Two families on one figure

Download occurrence data for both Ursidae and Mustelidae (weasels, otters, badgers) and display their genera side by side in a two-panel figure.

Objectives

- Create a `Figure` with two `Axis` objects: `fig[1, 1]` and `fig[1, 2]`.
- Run the full pipeline for each family and plot each in its own panel.
- Give each panel a title and set the same x-axis range so the panels are comparable.

Hints

- `linkxaxes!(ax1, ax2)` synchronises the x-axes of two panels after you create them.

- Use `xlims!(ax, low, high)` to set matching x limits on both axes.

1.10.3.4 Range duration histogram

For a taxon of your choice, create a histogram showing the distribution of genus-level range durations.

Objectives

- Compute `duration = first_appearance_datum - last_appearance_datum` for each genus.
- Plot a histogram with `hist!(ax, durations)`.
- Label axes clearly.
- Add a vertical line at the mean duration using `vlines!(ax, [mean_duration])`.

Hints

- You can control bin width with `hist!(ax, durations, bins = 10)`.
- Compute the mean as `sum(durations) / length(durations)`.

1.10.3.5 Export a summary table to CSV

Generate a tidy summary table for a taxon of your choice and save it.

Objectives

- Include columns: `genus`, `first_appearance_datum`, `last_appearance_datum`, `duration`, and `likely_extant` (true if `last_appearance_datum < 1.0`).
- Sort by duration descending (longest ranges first).
- Write the result to a file with `CSV.write("summary.csv", table_df)`.

Hints

- Install the CSV package with `] add CSV` in the package manager if needed, then using `CSV`.
- A Boolean column can be created with a dot comparison: `appearance_df.likely_extant = appearance_df.last_appearance_datum .< 1.0`.

1.10.3.6 Annotated figure with geological boundaries

Take the Ursidae range chart and add vertical dashed lines marking the Miocene/Pliocene boundary (5.3 Ma) and the Pliocene/Pleistocene boundary (2.6 Ma).

Objectives

- Add `vlines!(ax, [5.3, 2.6], color = :gray, linestyle = :dash)`.
- Annotate each line with a text label using `text!()`.
- Choose a figure size that keeps the labels readable.

Hints

- `text!(ax, x, y, text = "Pliocene/Pleistocene")` places text at (x, y) on the axis.
- `linestyle` can be `:dash`, `:dot`, or `:dashdot`.

1.10.3.7 Sensitivity to age model

The midpoint age is a simplification — a fossil could have any age within its uncertainty range. How much do the FAD and LAD estimates change depending on which age estimate you use?

Objectives

- Run the pipeline three times: using `max_ma`, `min_ma`, and the midpoint as the age estimate.
- Collect FAD and LAD for each approach into a comparison table.
- Plot three overlapping range charts (use `alpha = 0.4` for transparency) to visualise the spread.

Hints

- Only the line computing `df.midpoint_age` needs to change between runs.
- Use different colors for the three runs.

1.10.3.8 Occurrence count per genus

Do genera with more fossil occurrences tend to have longer estimated ranges?

Objectives

- Count occurrences per genus using split-apply-combine: `:genus => length => :n_occurrences`.
- Join the occurrence count into `appearance_df`: `innerjoin(appearance_df, counts_df, on = :genus)`.
- Create a scatter plot with `n_occurrences` on the x-axis and `duration` on the y-axis.

Hints

- `length` (without a dot) applied to a column in `combine` counts the number of rows in the group.
- Does a longer duration necessarily mean more occurrences? What might cause exceptions?

1.10.3.9 Exploring `pbdb_occurrences` parameters

The `pbdb_occurrences` function accepts many more keyword arguments than the three used in the tutorial.

Objectives

- Read the built-in documentation: type `?pbdb_occurrences` in the REPL.
- Download occurrences restricted to a specific geographic region (look for a `cc` or `continent` argument).
- Rerun the range chart pipeline on the filtered subset.
- Compare with the global result: which genera are missing? which remain?

Hints

- Use `show = "full"` to retrieve all available fields including geographic information.
- Start with a geographically rich taxon like Ursidae or Equidae to see clear regional patterns.

1.10.3.10 Oldest and youngest genera across five families

Build a figure showing, for each of five mammal families, which genus appeared first (oldest FAD) and which disappeared most recently (smallest LAD).

Objectives

- For each family, run the full pipeline and record the genus with the oldest FAD and the genus with the most recent LAD.
- Create a scatter or bar chart comparing the five families.

Hints

- A `for` loop over a vector of family names keeps the code compact: `for family in ["Ursidae", "Felidae", "Equidae", "Rhinocerotidae", "Cervidae"]`.
- Use `argmax` and `argmin` to locate the relevant genera in `appearance_df`.

2 Evaluating richness through time: standard binning method

2.1 Objectives

In the previous tutorial we drew a **range-through chart**: one horizontal line per genus, showing when each genus first and last appeared in the fossil record.

Now we want to collapse that picture into a single curve: **how many genera were present at any given time?**

The method we will use is called the **standard (sampled-in-bin) method with range-through interpolation**:

1. We divide geological time into named **stratigraphic intervals** (e.g. “Early Miocene”, “Late Pleistocene”).
2. For each genus we identify its **first appearance datum** (FAD) interval and its **last appearance datum** (LAD) interval.

3. We count a genus as present in any interval that falls *between* its FAD and LAD — even if no fossil from that genus was actually found in that interval. This is the *interpolation* step: we assume that if a genus was alive before and after a gap, it was alive during the gap too.
4. For each interval we count how many genera are present. That count is the **diversity** of the group at that time.

i Why interpolate?

The fossil record is incomplete. A genus that left no fossils in a particular interval was almost certainly still alive — it just was not preserved or has not yet been found. Interpolating between FAD and LAD is the standard way to correct for this *preservation bias*.

2.2 Environment setup

```
using DataFrames
using PaleobiologyDB
using PaleobiologyDB.Taxonomy
using Makie, CairoMakie

set_autocaching!(true)
```

2.3 Data acquisition

```
df = pbdb_occurrences(
    base_name = "Ursidae";
    show = "full",
    vocab = "pbdb"
)
```


	occurrence_no	record_type	reid_no	flags	collection_no	identified_name	
	Int64	String3	Int64?	Missing	Int64	String	
1	138737	occ	<i>missing</i>	<i>missing</i>	11798	Indarctos sinensis	...
2	138739	occ	<i>missing</i>	<i>missing</i>	11798	Ursinae indet.	...
3	150068	occ	<i>missing</i>	<i>missing</i>	13293	Ursus thibetanus	...
4	150069	occ	<i>missing</i>	<i>missing</i>	13293	Ailuropoda melanoleuca	...
5	177833	occ	<i>missing</i>	<i>missing</i>	16885	Parictis parvus n. sp.	...
6	177834	occ	<i>missing</i>	<i>missing</i>	16885	Parictis gilpini n. sp.	...
7	177925	occ	3434	<i>missing</i>	16907	Parictis personi	...
8	178031	occ	<i>missing</i>	<i>missing</i>	16919	Parictis parvus	...
9	178233	occ	<i>missing</i>	<i>missing</i>	16933	Parictis gilpini	...
10	178370	occ	<i>missing</i>	<i>missing</i>	16946	Parictis montanus	...
11	178414	occ	<i>missing</i>	<i>missing</i>	16958	Parictis gilpini	...
12	178466	occ	<i>missing</i>	<i>missing</i>	16960	Subparictis parvus	...
13	178509	occ	10552	<i>missing</i>	16961	Subparictis parvus	...
14	178553	occ	<i>missing</i>	<i>missing</i>	16962	Parictis sp.	...
15	178604	occ	<i>missing</i>	<i>missing</i>	16963	Parictis sp.	...
16	178641	occ	<i>missing</i>	<i>missing</i>	16964	Parictis sp.	...
17	178837	occ	<i>missing</i>	<i>missing</i>	16988	Parictis montanus	...
18	178937	occ	<i>missing</i>	<i>missing</i>	16998	Parictis gilpini	...
19	178953	occ	<i>missing</i>	<i>missing</i>	17006	Parictis major n. sp.	...
20	179118	occ	<i>missing</i>	<i>missing</i>	17014	Parictis sp.	...
21	179189	occ	<i>missing</i>	<i>missing</i>	17024	Parictis gilpini	...
22	179293	occ	<i>missing</i>	<i>missing</i>	17040	Parictis gilpini	...
23	179327	occ	3805	<i>missing</i>	17045	Parictis parvus	...
24	179507	occ	<i>missing</i>	<i>missing</i>	17082	Drassonax harpagops n. sp.	...
...	...	...	...	...	...	...	...

2.4 Data cleaning

```
df = dropmissing(df, [
    :genus,
    :early_interval,
    :max_ma,
    :min_ma
])
```

```
# Following runs both:
# - `df = Taxonomy.drop_unresolved_taxa(df, "genus")`
# - `df = Taxonomy.drop_unrecognized_taxa(df, :genus)`
df = Taxonomy.drop_unqualified_taxa(df, "genus")
```

	occurrence_no	record_type	reid_no	flags	collection_no	identified_name	
	Int64	String3	Int64?	Missing	Int64	String	
1	138737	occ	<i>missing</i>	<i>missing</i>	11798	Indarctos sinensis	...
2	150068	occ	<i>missing</i>	<i>missing</i>	13293	Ursus thibetanus	...
3	150069	occ	<i>missing</i>	<i>missing</i>	13293	Ailuropoda melanoleuca	...
4	177833	occ	<i>missing</i>	<i>missing</i>	16885	Parictis parvus n. sp.	...
5	177834	occ	<i>missing</i>	<i>missing</i>	16885	Parictis gilpini n. sp.	...
6	177925	occ	3434	<i>missing</i>	16907	Parictis personi	...
7	178031	occ	<i>missing</i>	<i>missing</i>	16919	Parictis parvus	...
8	178233	occ	<i>missing</i>	<i>missing</i>	16933	Parictis gilpini	...
9	178370	occ	<i>missing</i>	<i>missing</i>	16946	Parictis montanus	...
10	178414	occ	<i>missing</i>	<i>missing</i>	16958	Parictis gilpini	...
11	178466	occ	<i>missing</i>	<i>missing</i>	16960	Subparictis parvus	...
12	178509	occ	10552	<i>missing</i>	16961	Subparictis parvus	...
13	178553	occ	<i>missing</i>	<i>missing</i>	16962	Parictis sp.	...
14	178604	occ	<i>missing</i>	<i>missing</i>	16963	Parictis sp.	...
15	178641	occ	<i>missing</i>	<i>missing</i>	16964	Parictis sp.	...
16	178837	occ	<i>missing</i>	<i>missing</i>	16988	Parictis montanus	...
17	178937	occ	<i>missing</i>	<i>missing</i>	16998	Parictis gilpini	...
18	178953	occ	<i>missing</i>	<i>missing</i>	17006	Parictis major n. sp.	...
19	179118	occ	<i>missing</i>	<i>missing</i>	17014	Parictis sp.	...
20	179189	occ	<i>missing</i>	<i>missing</i>	17024	Parictis gilpini	...
21	179293	occ	<i>missing</i>	<i>missing</i>	17040	Parictis gilpini	...
22	179327	occ	3805	<i>missing</i>	17045	Parictis parvus	...
23	179507	occ	<i>missing</i>	<i>missing</i>	17082	Drassonax harpagops n. sp.	...
24	179963	occ	<i>missing</i>	<i>missing</i>	17147	Parictis gilpini	...
...	...	...	...	...	...	...	...

2.5 Computing midpoint ages

We will use midpoint ages to order intervals on the time axis, exactly as we did in the previous tutorial.

```
df.midpoint_age = (df.max_ma .+ df.min_ma) ./ 2
```

```
979-element Vector{Float64}:
```

```
 9.438
 0.39285000000000003
 0.39285000000000003
35.45
35.45
35.45
35.45
35.45
35.45
35.45

 0.00585
 0.07035
 1.677
 1.677
 1.29
 1.29
 0.00585
 0.00585
 0.4515
```

2.6 Building a table of stratigraphic intervals

We need a reference table listing every stratigraphic interval that appears in our data, with a representative age for each one so we can sort them from oldest to youngest.

We build it with the same split-apply-combine pattern you already know. We group by interval name and summarise the age bounds:

```
interval_df = combine(
  groupby(df, :early_interval),
  :max_ma => maximum => :interval_max_ma,
```

```

:min_ma => minimum => :interval_min_ma,
)

interval_df.interval_mid_ma = (interval_df.interval_max_ma .+ interval_df.interval_min_ma) ./ 2

interval_df = sort(interval_df, :interval_mid_ma, rev=true)

```

	early_interval	interval_max_ma	interval_min_ma	interval_mid_ma
	String31	Float64	Float64	Float64
1	Chadronian	37.0	33.9	35.45
2	Orellan	33.9	31.8	32.85
3	Early Oligocene	33.9	27.3	30.6
4	MP 23	33.9	27.3	30.6
5	MP 21	33.9	27.3	30.6
6	MP 22	33.9	27.3	30.6
7	Rupelian	33.9	27.3	30.6
8	Oligocene	33.9	23.04	28.47
9	Late Oligocene	27.3	23.04	25.17
10	Chattian	27.3	23.04	25.17
11	MP 28	27.3	23.04	25.17
12	Arikareean	29.5	18.5	24.0
13	late Late Arikareean	29.5	18.5	24.0
14	Aquitanian	23.04	20.45	21.745
15	MN 2	22.4	20.0	21.2
16	Harrisonian	23.1	18.5	20.8
17	MN 3	20.0	16.9	18.45
18	Orleanian	20.0	15.98	17.99
19	Early Hemingfordian	18.5	16.3	17.4
20	Hemingfordian	18.5	16.3	17.4
21	Early Miocene	23.04	11.63	17.335
22	MN 4	16.9	15.98	16.44
23	Early Barstovian	16.3	12.5	14.4
24	MN 5	15.98	12.75	14.365
...	...	...	...	...

`rev=true` puts the oldest intervals first (largest Ma value = furthest back in time). Take a look at what we have:

```
interval_df
```

	early_interval	interval_max_ma	interval_min_ma	interval_mid_ma
	String31	Float64	Float64	Float64
1	Chadronian	37.0	33.9	35.45
2	Orellan	33.9	31.8	32.85
3	Early Oligocene	33.9	27.3	30.6
4	MP 23	33.9	27.3	30.6
5	MP 21	33.9	27.3	30.6
6	MP 22	33.9	27.3	30.6
7	Rupelian	33.9	27.3	30.6
8	Oligocene	33.9	23.04	28.47
9	Late Oligocene	27.3	23.04	25.17
10	Chattian	27.3	23.04	25.17
11	MP 28	27.3	23.04	25.17
12	Arikareean	29.5	18.5	24.0
13	late Late Arikareean	29.5	18.5	24.0
14	Aquitanian	23.04	20.45	21.745
15	MN 2	22.4	20.0	21.2
16	Harrisonian	23.1	18.5	20.8
17	MN 3	20.0	16.9	18.45
18	Orleanian	20.0	15.98	17.99
19	Early Hemingfordian	18.5	16.3	17.4
20	Hemingfordian	18.5	16.3	17.4
21	Early Miocene	23.04	11.63	17.335
22	MN 4	16.9	15.98	16.44
23	Early Barstovian	16.3	12.5	14.4
24	MN 5	15.98	12.75	14.365
...	...	...	...	...

2.7 Finding FAD and LAD for each genus

Again, split-apply-combine. For each genus, take the maximum midpoint age across all its occurrences (= FAD, the oldest end of its range) and the minimum (= LAD, the most recent end).

```
genus_ranges = combine(
  groupby(df, :genus),
  :midpoint_age => maximum => :fad_ma,
  :midpoint_age => minimum => :lad_ma,
)
```

	genus	fad_ma	lad_ma
	String31	Float64	Float64
1	Indarctos	14.4	4.4665
2	Ursus	12.81	0.00585
3	Ailuropoda	3.09	0.07035
4	Parictis	35.45	32.85
5	Drassonax	32.85	32.85
6	Ursavus	24.0	6.9075
7	Cephalogale	30.6	17.4
8	Agriotherium	14.1865	3.05
9	Plionarctos	7.05	0.4515
10	Hemicyon	18.45	10.95
11	Plithocyon	18.215	10.6565
12	Phoberocyon	25.17	13.848
13	Arctodus	3.05	0.0645
14	Protarctos	4.4665	3.05
15	Tremarctos	3.05	0.0645
16	Ursulus	2.675	0.4515
17	Amphicynodon	30.6	25.17
18	Ballusia	18.45	13.848
19	Adelpharctos	30.6	25.17
20	Dinocyon	17.335	14.1865
21	Arctotherium	3.05	0.0645
22	Arctotherium (Pararctotherium)	1.29	0.0645
23	Phoberogale	30.6	21.2
24	Wangictis	30.6	30.6
...	...	...	...

2.8 Counting diversity per interval

This is the new piece. For each interval we need to count how many genera *range through* it.

We will need two tools we have not used before: `eachrow` and `count`.

2.8.1 Iterating over rows with `eachrow`

In the previous tutorial our loop used an index `i` to refer to both the row and the y-position:

```
for i in 1:nrow(some_df)
  val = some_df.column[i]
  ...
end
```

When we only need the *values* in each row and do not need the row number, there is a cleaner way: `eachrow(df)` produces the rows of a DataFrame one at a time, each as a named object you can access by column name:

```
for row in eachrow(some_df)
  val = row.column
  ...
end
```

Both approaches do the same job; `eachrow` is tidier when the index itself is not needed for anything.

2.8.2 Counting with a condition: `count`

`count(condition, collection)` goes through every item in `collection`, applies `condition` to it, and returns how many times `condition` was true.

```
count(x -> x > 0, [-1, 2, 3, -4, 5])  # → 3
```

The `x -> x > 0` part is an **anonymous function**: a small, unnamed function defined on the spot. `x` is its argument; `x > 0` is what it computes. You can read it as “for each `x`, check whether `x > 0`”.

i Anonymous functions

`x -> expression` is Julia's syntax for a function with no name. It is useful when the function is short enough that giving it a name and a separate `function ... end` block would just be clutter.

2.8.3 Building a vector incrementally with `push!`

We are going to compute one diversity number per interval and collect them into a vector. We start with an empty vector and add to it one element at a time using `push!`:

```
v = Int[]           # an empty vector that will hold integers
push!(v, 10)         # v is now [10]
push!(v, 7)          # v is now [10, 7]
```

i The `!` convention

In Julia, a `!` at the end of a function name signals that the function *modifies* one of its arguments in place. `push!(v, x)` changes `v` itself, rather than returning a new vector.

2.8.4 Putting it together

```
diversity = Int[]

for interval_row in eachrow(interval_df)
    n = count(
        g -> g.fad_ma >= interval_row.interval_mid_ma >= g.lad_ma,
        eachrow(genus_ranges)
    )
    push!(diversity, n)
end

interval_df.diversity = diversity
```


70-element Vector{Int64}:

1
2
7
7
7
7
7
5
7
7

8
8
8
8
6
6
6
5
1

Walk through what happens on each pass of the loop:

- `interval_row` is one row from `interval_df` — one stratigraphic interval.
- `count(...)` goes through every row `g` of `genus_ranges` and checks whether that genus's FAD and LAD bracket the current interval's midpoint age. The expression `g.fad_ma >= interval_row.interval_mid_ma >= g.lad_ma` is Julia's **chained comparison** — equivalent to `g.fad_ma >= interval_row.interval_mid_ma && interval_row.interval_mid_ma >= g.lad_ma`.
- `push!(diversity, n)` appends that count to our growing vector.

After the loop, `interval_df.diversity` holds the diversity value for every interval.

2.9 Plotting the diversity curve

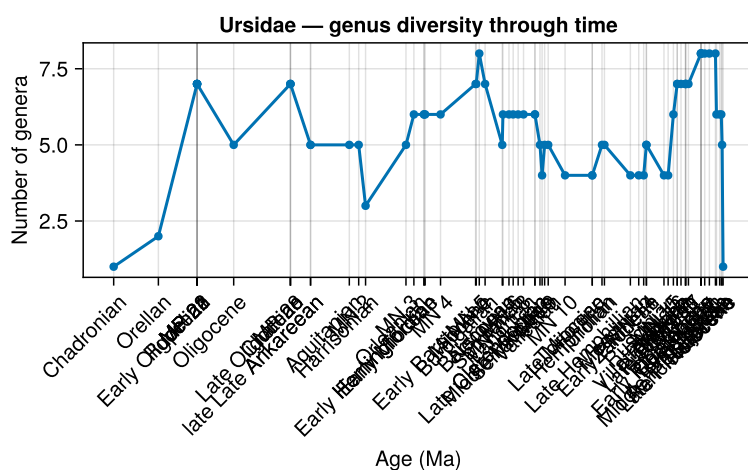
You already know all the pieces here. We use `lines!` and `scatter!` together — the line connects the points, the scatter markers make the individual interval values visible.

The x-axis uses the interval midpoint ages, with interval names as tick labels. `xticklabelrotation = /4` tilts the labels 45° so they do not overlap.

```
fig = Figure()
ax = Axis(
    fig[1, 1],
    xlabel = "Age (Ma)",
    ylabel = "Number of genera",
    title = "Ursidae - genus diversity through time",
    xticks = (interval_df.interval_mid_ma, interval_df.early_interval),
    xticklabelrotation = /4,
    xreversed = true,
)

lines!(ax, interval_df.interval_mid_ma, interval_df.diversity, linewidth = 2)
scatter!(ax, interval_df.interval_mid_ma, interval_df.diversity, markersize = 8)

fig
```



2.10 Complete program

```
using DataFrames
using PaleobiologyDB
using PaleobiologyDB.Taxonomy
using Makie, CairoMakie

set_autocaching!(true)

df = pbdb_occurrences(
    base_name = "Ursidae";
    show = "full",
    vocab = "pbdb"
)

df = dropmissing(df, [
    :genus,
    :early_interval,
    :max_ma,
    :min_ma
])

# Following runs both:
# - `df = Taxonomy.drop_unresolved_taxa(df, "genus")`
# - `df = Taxonomy.drop_unrecognized_taxa(df, :genus)`
df = Taxonomy.drop_unqualified_taxa(df, "genus")

df.midpoint_age = (df.max_ma .+ df.min_ma) ./ 2

interval_df = combine(
    groupby(df, :early_interval),
    :max_ma => maximum => :interval_max_ma,
    :min_ma => minimum => :interval_min_ma,
)

interval_df.interval_mid_ma = (interval_df.interval_max_ma .+ interval_df.interval_min_ma) ./ 2
```

```

interval_df = sort(interval_df, :interval_mid_ma, rev=true)

interval_df

genus_ranges = combine(
  groupby(df, :genus),
  :midpoint_age => maximum => :fad_ma,
  :midpoint_age => minimum => :lad_ma,
)

diversity = Int[]

for interval_row in eachrow(interval_df)
  n = count(
    g -> g.fad_ma >= interval_row.interval_mid_ma >= g.lad_ma,
    eachrow(genus_ranges)
  )
  push!(diversity, n)
end

interval_df.diversity = diversity

fig = Figure()
ax = Axis(
  fig[1, 1],
  xlabel = "Age (Ma)",
  ylabel = "Number of genera",
  title = "Ursidae - genus diversity through time",
  xticks = (interval_df.interval_mid_ma, interval_df.early_interval),
  xticklabelrotation = /4,
  xreversed = true,
)

lines!(ax, interval_df.interval_mid_ma, interval_df.diversity, linewidth = 2)
scatter!(ax, interval_df.interval_mid_ma, interval_df.diversity, markersize = 8)

fig

```

2.11 Exercises: Evaluating richness through time — standard binning method

2.11.1 Julia concepts and mechanisms

For each snippet, predict the output before running it in the REPL.

2.11.1.1 Part A: Anonymous functions

1. A named anonymous function:

```
f = x -> x * 2  
println(f(5))
```

i Answer

10

`x -> x * 2` defines an unnamed function; assigning it to `f` lets you call it like any other function.

2. Testing a condition:

```
f = x -> x > 10  
println(f(5))  
println(f(15))
```

i Answer

false

true

3. Anonymous function inline in `count`:

```
println(count(x -> x > 0, [-1, 2, -3, 4, 5]))
```

i Answer

3

`count` applies the condition to every element and tallies the `true` results. The positive values are 2, 4, 5 — three of them.

4. Counting strings by length:

```
names = ["cat", "dog", "bear", "fox"]
println(count(n -> length(n) == 3, names))
```

i Answer

3

“cat”, “dog”, and “fox” all have length 3; “bear” has length 4.

5. Threshold count:

```
ages = [5.0, 12.0, 3.0, 7.0, 20.0]
println(count(a -> a >= 7.0, ages))
```

i Answer

3

Values 7.0: 12.0, 7.0, 20.0.

6. Anonymous function with arithmetic:

```
f = x -> x^2 + 1
println(f(3))
println(f(0))
```

i Answer

10

1

$f(3) = 3^2 + 1 = 10$; $f(0) = 0 + 1 = 1$.

2.11.1.2 Part B: push! and building vectors incrementally

1. Basic push!:

```
v = Int[]  
push!(v, 10)  
push!(v, 20)  
push!(v, 30)  
println(v)
```

i Answer

[10, 20, 30]

Int[] starts as an empty integer vector. Each push! appends one element in place.

2. push! inside a loop:

```
v = Int[]  
for i in 1:5  
    push!(v, i * 2)  
end  
println(v)
```

i Answer

[2, 4, 6, 8, 10]

3. Length after pushing:

```
ages = Float64[]  
push!(ages, 3.5)  
push!(ages, 7.2)  
println(length(ages))
```

i Answer

2

4. Conditional push!:

```
v = Int[]
for i in 1:6
    if i % 2 == 0
        push!(v, i)
    end
end
println(v)
```

i Answer

[2, 4, 6]

Only even values of i satisfy $i \% 2 == 0$.

5. count with a bracketing condition:

```
ages = [5.0, 15.0, 25.0, 35.0, 45.0]
println(count(a -> 10.0 <= a <= 30.0, ages))
```

i Answer

2

15.0 and 25.0 satisfy $10.0 \leq a \leq 30.0$; the others do not.

6. Counting even numbers:

```
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
println(count(x -> x % 2 == 0, data))
```

i Answer

5

Even numbers from 1 to 10: 2, 4, 6, 8, 10.

2.11.1.3 Part C: eachrow iteration

1. Iterating over rows by name:

```
using DataFrames
df = DataFrame(name = ["A", "B", "C"], val = [1, 2, 3])
for row in eachrow(df)
```



```
println(row.name, ": ", row.val)
end
```

i Answer

A: 1

B: 2

C: 3

`eachrow` yields each row as a named object;
`row.name` and `row.val` access fields by column name
instead of numeric index.

2. Computing with `eachrow` and `push!`:

```
using DataFrames
df = DataFrame(fad = [10.0, 20.0, 30.0], lad = [2.0, 5.0, 10.0])
durations = Float64[]
for row in eachrow(df)
    push!(durations, row.fad - row.lad)
end
println(durations)
```

i Answer

[8.0, 15.0, 20.0]

Row-by-row: 10-2=8, 20-5=15, 30-10=20.

3. `count` over `eachrow`:

```
using DataFrames
genus_ranges = DataFrame(
    fad_ma = [50.0, 30.0, 10.0],
    lad_ma = [20.0, 5.0, 1.0],
)
n = count(g -> g.fad_ma >= 25.0 >= g.lad_ma, eachrow(genus_ranges))
println(n)
```

i Answer

2

Row 1: 50 25 20 → true. Row 2: 30 25 5
→ true. Row 3: 10 25? No → false.

4. `eachrow` vs. index loop — same output?

```
using DataFrames
df = DataFrame(x = [10, 20, 30])

for row in eachrow(df)
    print(row.x, " ")
end
println()

for i in 1:nrow(df)
    print(df.x[i], " ")
end
println()
```

i Answer

10 20 30

10 20 30

Both loops produce the same output. `eachrow` is tidier when you do not need the row number.

5. Accumulating a sum with `push!` then summing:

```
v = Float64[]
for i in 1:4
    push!(v, i * 1.5)
end
println(v)
println(sum(v))
```

i Answer

[1.5, 3.0, 4.5, 6.0]

15.0

```
1.5 + 3.0 + 4.5 + 6.0 = 15.0.
```

6. How many rows does `eachrow` iterate over?

```
using DataFrames
df = DataFrame(a = 1:7, b = 11:17)
n = 0
for row in eachrow(df)
    n += 1
end
println(n)
```

i Answer

7

`eachrow` iterates once per row; `nrow(df) = 7`.

2.11.1.4 Part D: Chained comparisons

1. A simple chained comparison:

```
x = 15.0
println(10.0 <= x <= 20.0)
```

i Answer

true

2. Chained comparison that fails at the second step:

```
x = 25.0
println(10.0 <= x <= 20.0)
```

i Answer

false

10.0 ≤ 25.0 is true, but 25.0 ≤ 20.0 is false, so the whole expression is false.

3. Chained comparison as an anonymous function:

```
in_range = x -> 10.0 <= x <= 30.0
println(in_range(20.0))
println(in_range(35.0))
```

i Answer

```
true
false
```

4. Three-variable chain:

```
a, b, c = 5.0, 10.0, 15.0
println(a <= b <= c)
println(c >= b >= a)
```

i Answer

```
true
true
Both orderings express the same relationship: a ≤ b ≤ c.
```

5. Chain that fails at the first step:

```
println(5 < 3 < 10)
```

i Answer

```
false
5 < 3 is false; Julia short-circuits and the whole
expression is false.
```

6. FAD/LAD bracket check:

```
fad = 30.0
lad = 5.0
interval_mid = 15.0
println(fad >= interval_mid >= lad)
```

i Answer

```
true
30.0 15.0 and 15.0 5.0: the interval midpoint
falls within the genus range.
```

7. Difference between `>=` at the boundary:

```
fad = 15.0
lad = 5.0
interval_mid = 15.0
println(fad >= interval_mid >= lad) # non-strict
println(fad > interval_mid > lad) # strict
```

i Answer

```
true
false
```

With `>=`, a genus whose FAD equals the interval midpoint is counted as present. With `>`, it is not — the strict version requires the FAD to be *older* than the midpoint. (We will return to this distinction in the next tutorial.)

2.11.1.5 Part E: DataFrame constructor and plotting

1. Building a DataFrame from existing vectors:

```
using DataFrames
boundary_ma = [10.0, 20.0, 30.0]
diversity    = [3, 5, 4]
df = DataFrame(boundary_ma = boundary_ma, diversity = diversity)
println(nrow(df))
println(df.diversity[2])
```

i Answer

```
3
5
DataFrame{col = vec, ...} constructs a
```

DataFrame directly from named vectors.
df.diversity[2] is the second element: 5.

2. What does `xticks = (positions, labels)` do?

```
positions = [1.0, 2.0, 3.0]
labels    = ["Early", "Middle", "Late"]
# Axis(..., xticks = (positions, labels))
```

No running needed — describe the effect.

i Answer

Tick marks are placed at x-coordinates 1.0, 2.0, 3.0, with the text labels “Early”, “Middle”, “Late” shown at those positions. This replaces the automatic numeric ticks with human-readable interval names.

3. The value of `/4`:

```
println( / 4)
```

i Answer

0.7853981633974483

This is 45° in radians — used as `xticklabelrotation = /4` to tilt axis labels so they do not overlap.

4. What do `lines!` and `scatter!` produce together?

```
x = [1.0, 2.0, 3.0, 4.0, 5.0]
y = [2,   5,   3,   7,   4 ]
# lines!(ax, x, y)
# scatter!(ax, x, y)
```

No running needed — describe the combined result.

i Answer

`lines!` draws a continuous line connecting the (x, y) points in order. `scatter!` draws a marker at each (x, y) point. Together they produce a *connected scatter plot*: the line shows the overall shape of the curve, and the markers indicate the individual measured values.

5. Full diversity counting pattern:

```
using DataFrames
genus_ranges = DataFrame(
  genus = ["A", "B", "C", "D"],
  fad_ma = [40.0, 30.0, 20.0, 10.0],
  lad_ma = [15.0, 5.0, 2.0, 1.0],
)
interval_mid = 18.0
n = count(g -> g.fad_ma >= interval_mid >= g.lad_ma, eachrow(genus_ranges))
println(n)
```

i Answer

3
A: 40 18 15 → true. B: 30 18 5 → true. C:
20 18 2 → true. D: 10 18? No → false.

2.11.2 Practical shorts

Assume you have completed the tutorial and have `df`, `interval_df`, `genus_ranges`, and `interval_df.diversity` available in your REPL.

1. How many distinct stratigraphic intervals are in `interval_df`? Print the result with `nrow(interval_df)`.
2. Print the interval with the largest `interval_max_ma` (the oldest interval). `interval_df[1, :]` after sorting by `interval_mid_ma` descending.

3. Compute the duration of each stratigraphic interval and add it as a column: `interval_df.duration = interval_df.interval_max_ma .- interval_df.interval_min_ma`
Which interval is longest?
4. Which interval has the highest diversity? `interval_df.early_interval[argmax(interval_df.diversity)]`
5. Which interval has the lowest diversity?
6. Compute the mean diversity across all intervals:
`sum(interval_df.diversity) / nrow(interval_df)`
7. Count how many genera have a FAD older than 20 Ma:
`count(g -> g.fad_ma > 20.0, eachrow(genus_ranges))`
8. Count how many genera were present (ranged through) during an interval of your choice. Find that interval's midpoint in `interval_df`, then use: `count(g -> g.fad_ma >= mid >= g.lad_ma, eachrow(genus_ranges))`
9. How many genera have both `fad_ma > 20.0` and `lad_ma < 5.0` (archaic genera, long gone before the present)?
`count(g -> g.fad_ma > 20.0 && g.lad_ma < 5.0, eachrow(genus_ranges))`
10. What is the total time span covered by `interval_df`?
`maximum(interval_df.interval_max_ma) - minimum(interval_df.interval_min_ma)`
11. Change `linewidth = 2` to `linewidth = 4` and `markersize = 8` to `markersize = 12` in the diversity curve plot, then redraw.
12. Change `xticklabelrotation = /4` to `xticklabelrotation = /3`. What visual difference does this make?
13. Add a horizontal reference line at `diversity = 5`:
`hlines!(ax, [5], color = :gray, linestyle = :dash)`
14. Rerun the full pipeline for Felidae (cats). How does the diversity curve differ from Ursidae?
15. Print the names and midpoint ages of all intervals where diversity is greater than the mean diversity.
16. Use `eachrow` to write a loop that prints "Genus X: FAD=Y Ma, LAD=Z Ma" for every genus in `genus_ranges`.

17. Replace the `xticks` interval names with just the numeric midpoint ages. Change `xticks = (interval_df.interval_mid_ma, interval_df.early_interval)` to `xticks = interval_df.interval_mid_ma`. How does the x-axis change?
18. Find the interval with the sharpest diversity drop from the preceding interval. Compute `diffs = diff(interval_df.diversity)` and find the index of the minimum.
19. Compute diversity per interval for a second taxon and store it in a second column of `interval_df` (e.g., `interval_df.diversity_felidae = ...`). Then plot both curves on the same axis in different colors.
20. Rewrite the diversity counting loop using an index loop (`for i in 1:nrow(interval_df)`) instead of `eachrow`. Do you get the same result?
21. Instead of using `interval_df.interval_mid_ma` as the midpoint, recompute the interval reference table using the original `max_ma` column directly. Does this change the diversity estimates?
22. How does the diversity count change if you use `strict >` instead of `>=` in the bracketing condition? `count(g -> g.fad_ma > interval_row.interval_mid_ma > g.lad_ma, ...)` For which intervals, if any, does the result differ?
23. The diversity curve uses `interval_mid_ma` on the x-axis. Replace it with `interval_df.interval_max_ma` (older edge of each interval) and redraw. What shifts in the plot?
24. Add scatter markers colored by diversity using a colormap:

```
scatter!(ax, interval_df.interval_mid_ma, interval_df.diversity,
         color = interval_df.diversity, colormap = :viridis,
         markersize = 12)
```
25. Change `rev=true` to `rev=false` in the `sort` that orders `interval_df`. How does the diversity curve look? Is the time axis still correct?

26. What happens if you call `push!(diversity, n)` outside the loop (after it ends) instead of inside? Walk through the logic without running it first, then test.
 27. Compute a three-interval rolling mean of diversity. For each interval `i` from 2 to `nrow - 1`: `mean = (diversity[i-1] + diversity[i] + diversity[i+1]) / 3` Collect these into a vector and plot it alongside the original curve.
 28. Export `interval_df` (with the diversity column) to a CSV file. Install and import CSV (using CSV), then: `CSV.write("ursidae_diversity.csv", interval_df)`
 29. Try counting diversity using `eachrow(genus_ranges)` vs. iterating `genus_ranges` with a numeric index and comparing the counts. Confirm they match for at least one interval.
 30. Check: does every interval in `interval_df` have at least one genus present? Print any interval where diversity is 0.
-

2.11.3 Creative programming

2.11.3.1 Diversity profile comparison for two families

Compute and plot the sampled-in-bin diversity curve for two closely related families (for example Ursidae and Procyonidae, or Felidae and Hyaenidae) on the same figure.

Objectives

- Run the full pipeline for both families.
- Plot both diversity curves on the same **Axis** using different colors.
- Add a legend identifying each curve.
- Write one or two sentences describing how the diversity histories differ.

Hints

- `lines!(ax, x, y, label = "Ursidae", color = :steelblue)` adds a labelled series.
- `axislegend(ax)` draws a legend automatically from the labels.

2.11.3.2 Regional diversity curves

Use the geographic fields in the PBDB occurrence data to compute separate diversity curves for occurrences from North America versus Europe.

Objectives

- Filter `df` to two geographic subsets after downloading.
- Run the full pipeline separately for each region.
- Plot the two curves on the same figure and compare.

Hints

- The `cc` field in the occurrence data holds the continent code. Filter with `df[df.cc .== "NOA", :]` for North America (code may vary — check the data).
- Both subsets share the same `interval_df`; only the diversity counting step changes.

2.11.3.3 Sampling-corrected diversity

The diversity curve is affected by how well-sampled each interval is. A time interval with more fossil collections will tend to show higher diversity, even if diversity was not actually higher.

Objectives

- Add a column `n_occurrences` to `interval_df` counting how many raw occurrence records fall within each interval.
- Plot both raw diversity and occurrence count as two lines on a dual-axis figure.
- Comment on intervals where the two curves diverge.

Hints

- Use `split-apply-combine` on `df` grouped by `:early_interval` with `:early_interval => length => :n_occurrences` to count occurrences per interval.
- A second y-axis can be added in Makie with `Axis` placed at `fig[1, 1]` with a `yaxisposition = :right` argument.

2.11.3.4 Origination and extinction counts

At each stratigraphic interval, some genera appear for the first time (originations) and some disappear (extinctions). Plot these alongside the diversity curve.

Objectives

- Define an origination as a genus whose FAD falls within an interval's age range.
- Define an extinction as a genus whose LAD falls within an interval's age range.
- For each interval, count originations and extinctions.
- Plot diversity, originations, and extinctions on the same figure.

Hints

- `count(g -> interval_min <= g.fad_ma <= interval_max, eachrow(genus_ranges))` counts originations for one interval.
- Use different line colors and a legend to distinguish the three curves.

2.11.3.5 Custom equal-time bins

Instead of using PBDB stratigraphic intervals, divide time into equal 5-Ma bins and recompute the diversity curve.

Objectives

- Create your own `interval_df` with 5-Ma bins spanning the dataset's age range.
- Rerun the diversity counting step with these custom bins.
- Plot the custom-bin curve alongside the PBDB-interval curve and compare the shapes.

Hints

- You can create evenly spaced bin edges with: `bin_edges = 0.0:5.0:60.0`.
- The midpoint of each bin is `(left_edge + right_edge) / 2`.
- Build a DataFrame from these vectors using `DataFrame(interval_max_ma = ..., interval_min_ma = ..., interval_mid_ma = ...)`.

2.11.3.6 Color-coded diversity scatter

Map the diversity value to color so that each scatter marker is colored by how diverse the fauna was at that time.

Objectives

- Replace the uniform scatter markers with color-mapped ones using a viridis or plasma colormap.
- Add a Colorbar to the figure showing the diversity scale.
- Try at least two different colormaps and choose the one you find most readable.

Hints

- `scatter!(ax, x, y, color = diversity, colormap = :plasma, markersize = 12)` maps the diversity vector to the colormap.
- `Colorbar(fig[1, 2], colormap = :plasma, limits = (0, maximum(diversity)))` adds a vertical colorbar next to the plot.

2.11.3.7 Persistence through time

Instead of raw diversity, compute the *persistence fraction* at each interval: what fraction of the genera present in interval `i` are still present in interval `i+1`?

Objectives

- For each pair of adjacent intervals, compute how many genera span both.

- Express this as a fraction of the diversity in the older interval.
- Plot the persistence fraction as a curve.
- Identify intervals with the lowest persistence (largest faunal turnovers).

Hints

- You will need the diversity counts for both intervals simultaneously; iterate with an index loop rather than `eachrow`.
- A persistence fraction close to 1.0 means nearly all genera survived; close to 0 means near-total turnover.

2.11.3.8 Diversity for a deep-time taxon

Apply the pipeline to a group with a much longer fossil record than Ursidae — for example Crocodilia or Testudines (turtles) — spanning hundreds of millions of years.

Objectives

- Download and clean the data; note how many intervals are returned.
- Run the diversity pipeline and plot the result.
- Identify the interval of peak diversity.
- Describe one paleobiological event visible in the diversity curve.

Hints

- Very old data may have large age uncertainties; the midpoint calculation still applies.
- The diversity curve may be noisy at the oldest end where sampling is sparse.

2.11.3.9 The Makie gallery

Browse the Makie documentation examples gallery and find one chart type you have not used yet — for example a violin plot, heatmap, ridgeline, or step plot.

Objectives

- Choose a plot type from the gallery.
- Adapt it to display something meaningful about the Ursidae dataset.
- Add appropriate labels and a title.

Hints

- The Makie documentation is at docs.makie.org.
- You already know how to build the Ursidae data; the challenge is choosing the right visual encoding and adapting the syntax from the example.

2.11.3.10 Comparing diversity methods across five families

For five mammal families of your choice, compute the sampled-in-bin diversity at its peak interval and compare across families.

Objectives

- Run the full pipeline for each family.
- Record the peak diversity and the interval at which it occurs.
- Create a bar chart comparing peak diversities across the five families.
- Label each bar with the family name and the interval.

Hints

- A `for` loop over family names keeps the code compact.
- `barplot!(ax, 1:5, peak_diversities)` draws the bars; use `ax.xticks` to add family name labels.

3 Evaluating richness through time: range-through method

3.1 Objectives

In the previous tutorial we counted how many genera were *present* inside each stratigraphic interval, interpolating across gaps.

This tutorial uses a subtly different approach: the **range-through method**. Instead of asking “which genera were present *in* this interval?”, we ask “which genera were present *across* this interval boundary?” — meaning: observed both *before* and *after* a given point in time.

A genus counts toward the diversity at a boundary only if it has occurrences **both** older and younger than that boundary. This is a stricter requirement, and it means we are measuring taxa that genuinely co-existed in time, rather than taxa whose ranges merely happen to span an interval by interpolation.

i Range-through vs sampled-in-bin

The **sampled-in-bin** method (previous tutorial) counts every genus whose range spans an interval, including genera whose ranges only *touch* the interval at one end. The **range-through** method counts only genera that have documented occurrences on *both sides* of a boundary. It is more conservative but less susceptible to edge effects at the FAD or LAD.

The change to the code from the previous tutorial is small: one condition in the `count` call.

3.2 Environment setup

```
using DataFrames
using PaleobiologyDB
using PaleobiologyDB.Taxonomy
using Makie, CairoMakie

set_autocaching!(true)
```


3.3 Data acquisition

```
df = pbdb_occurrences(
    base_name = "Ursidae";
    show = "full",
    vocab = "pbdb"
)
```

	occurrence_no	record_type	reid_no	flags	collection_no	identified_name	
	Int64	String3	Int64?	Missing	Int64	String	
1	138737	occ	<i>missing</i>	<i>missing</i>	11798	Indarctos sinensis	...
2	138739	occ	<i>missing</i>	<i>missing</i>	11798	Ursinae indet.	...
3	150068	occ	<i>missing</i>	<i>missing</i>	13293	Ursus thibetanus	...
4	150069	occ	<i>missing</i>	<i>missing</i>	13293	Ailuropoda melanoleuca	...
5	177833	occ	<i>missing</i>	<i>missing</i>	16885	Parictis parvus n. sp.	...
6	177834	occ	<i>missing</i>	<i>missing</i>	16885	Parictis gilpini n. sp.	...
7	177925	occ	3434	<i>missing</i>	16907	Parictis personi	...
8	178031	occ	<i>missing</i>	<i>missing</i>	16919	Parictis parvus	...
9	178233	occ	<i>missing</i>	<i>missing</i>	16933	Parictis gilpini	...
10	178370	occ	<i>missing</i>	<i>missing</i>	16946	Parictis montanus	...
11	178414	occ	<i>missing</i>	<i>missing</i>	16958	Parictis gilpini	...
12	178466	occ	<i>missing</i>	<i>missing</i>	16960	Subparictis parvus	...
13	178509	occ	10552	<i>missing</i>	16961	Subparictis parvus	...
14	178553	occ	<i>missing</i>	<i>missing</i>	16962	Parictis sp.	...
15	178604	occ	<i>missing</i>	<i>missing</i>	16963	Parictis sp.	...
16	178641	occ	<i>missing</i>	<i>missing</i>	16964	Parictis sp.	...
17	178837	occ	<i>missing</i>	<i>missing</i>	16988	Parictis montanus	...
18	178937	occ	<i>missing</i>	<i>missing</i>	16998	Parictis gilpini	...
19	178953	occ	<i>missing</i>	<i>missing</i>	17006	Parictis major n. sp.	...
20	179118	occ	<i>missing</i>	<i>missing</i>	17014	Parictis sp.	...
21	179189	occ	<i>missing</i>	<i>missing</i>	17024	Parictis gilpini	...
22	179293	occ	<i>missing</i>	<i>missing</i>	17040	Parictis gilpini	...
23	179327	occ	3805	<i>missing</i>	17045	Parictis parvus	...
24	179507	occ	<i>missing</i>	<i>missing</i>	17082	Drassonax harpagops n. sp.	...
...	...	...	...	...	...	...	...

3.4 Data cleaning

```
df = dropmissing(df, [
    :genus,
    :early_interval,
    :max_ma,
    :min_ma
])
df = Taxonomy.drop_unqualified_taxa(df, "genus")
```

	occurrence_no	record_type	reid_no	flags	collection_no	identified_name	
	Int64	String3	Int64?	Missing	Int64	String	
1	138737	occ	<i>missing</i>	<i>missing</i>	11798	Indarctos sinensis	...
2	150068	occ	<i>missing</i>	<i>missing</i>	13293	Ursus thibetanus	...
3	150069	occ	<i>missing</i>	<i>missing</i>	13293	Ailuropoda melanoleuca	...
4	177833	occ	<i>missing</i>	<i>missing</i>	16885	Parictis parvus n. sp.	...
5	177834	occ	<i>missing</i>	<i>missing</i>	16885	Parictis gilpini n. sp.	...
6	177925	occ	3434	<i>missing</i>	16907	Parictis personi	...
7	178031	occ	<i>missing</i>	<i>missing</i>	16919	Parictis parvus	...
8	178233	occ	<i>missing</i>	<i>missing</i>	16933	Parictis gilpini	...
9	178370	occ	<i>missing</i>	<i>missing</i>	16946	Parictis montanus	...
10	178414	occ	<i>missing</i>	<i>missing</i>	16958	Parictis gilpini	...
11	178466	occ	<i>missing</i>	<i>missing</i>	16960	Subparictis parvus	...
12	178509	occ	10552	<i>missing</i>	16961	Subparictis parvus	...
13	178553	occ	<i>missing</i>	<i>missing</i>	16962	Parictis sp.	...
14	178604	occ	<i>missing</i>	<i>missing</i>	16963	Parictis sp.	...
15	178641	occ	<i>missing</i>	<i>missing</i>	16964	Parictis sp.	...
16	178837	occ	<i>missing</i>	<i>missing</i>	16988	Parictis montanus	...
17	178937	occ	<i>missing</i>	<i>missing</i>	16998	Parictis gilpini	...
18	178953	occ	<i>missing</i>	<i>missing</i>	17006	Parictis major n. sp.	...
19	179118	occ	<i>missing</i>	<i>missing</i>	17014	Parictis sp.	...
20	179189	occ	<i>missing</i>	<i>missing</i>	17024	Parictis gilpini	...
21	179293	occ	<i>missing</i>	<i>missing</i>	17040	Parictis gilpini	...
22	179327	occ	3805	<i>missing</i>	17045	Parictis parvus	...
23	179507	occ	<i>missing</i>	<i>missing</i>	17082	Drassonax harpagops n. sp.	...
24	179963	occ	<i>missing</i>	<i>missing</i>	17147	Parictis gilpini	...
...	...	...	...	...	...	...	...

3.5 Computing midpoint ages

```
df.midpoint_age = (df.max_ma .+ df.min_ma) ./ 2
```

```
979-element Vector{Float64}:
```

```
 9.438
 0.39285000000000003
 0.39285000000000003
35.45
35.45
35.45
35.45
35.45
35.45
35.45
0.00585
0.07035
1.677
1.677
1.29
1.29
0.00585
0.00585
0.4515
```

3.6 Building the interval table and genus ranges

```
interval_df = combine(
  groupby(df, :early_interval),
  :max_ma => maximum => :interval_max_ma,
  :min_ma => minimum => :interval_min_ma,
)

interval_df.interval_mid_ma = (
```

```

interval_df.interval_max_ma .+ interval_df.interval_min_ma
) ./ 2

interval_df = sort(interval_df, :interval_mid_ma, rev=true)

genus_ranges = combine(
  groupby(df, :genus),
  :midpoint_age => maximum => :fad_ma,
  :midpoint_age => minimum => :lad_ma,
)

```

	genus	fad_ma	lad_ma
	String31	Float64	Float64
1	Indarctos	14.4	4.4665
2	Ursus	12.81	0.00585
3	Ailuropoda	3.09	0.07035
4	Parictis	35.45	32.85
5	Drassonax	32.85	32.85
6	Ursavus	24.0	6.9075
7	Cephalogale	30.6	17.4
8	Agriotherium	14.1865	3.05
9	Plionarctos	7.05	0.4515
10	Hemicyon	18.45	10.95
11	Plithocyon	18.215	10.6565
12	Phoberocyon	25.17	13.848
13	Arctodus	3.05	0.0645
14	Protarctos	4.4665	3.05
15	Tremarctos	3.05	0.0645
16	Ursulus	2.675	0.4515
17	Amphicynodon	30.6	25.17
18	Ballusia	18.45	13.848
19	Adelpharctos	30.6	25.17
20	Dinocyon	17.335	14.1865
21	Arctotherium	3.05	0.0645
22	Arctotherium (Pararctotherium)	1.29	0.0645
23	Phoberogale	30.6	21.2
24	Wangictis	30.6	30.6
...	...	...	...

3.7 Measuring diversity at interval *boundaries*

In the previous tutorial the diversity score was computed *inside* an interval — we checked whether a genus’s range spanned the interval’s midpoint.

Here we compute diversity at the *boundary* between two successive intervals. A boundary sits between interval `i` (older) and interval `i+1` (younger). Its age is the shared edge: the younger end of interval `i`, which equals the older end of interval `i+1`.

A genus counts at that boundary if and only if:

- its FAD is **older** than the boundary (it was already present before), **and**
- its LAD is **younger** than the boundary (it persisted after).

In terms of the midpoint ages we have stored:

```
g.fad_ma > boundary_ma > g.lad_ma
```

Both inequalities must be **strict** (the genus must be observed on *both sides*, not merely *at* the boundary).

Why strict inequalities?

A genus whose FAD equals the boundary age was first seen *at* the boundary, not before it. We cannot say it was present on the older side. Strict `>` ensures we only count genera with confirmed occurrences on both sides.

3.7.1 Extracting the boundary ages

Each pair of adjacent intervals in `interval_df` shares a boundary. There are `nrow(interval_df) - 1` boundaries — one fewer than the number of intervals.

We extract them using an **index range with a step**. You have already seen `1:n` (every integer from 1 to `n`). Julia ranges can also specify a step:

```
1:2:9    # → 1, 3, 5, 7, 9 (step of 2)
```

We do not need a custom step here, but we do need to stop one row short of the end: `1:nrow(interval_df)-1` gives the index of every interval except the last.

We use the **younger end** of each interval (`interval_min_ma`) as the boundary age, which equals the older end of the next interval:

```
boundary_ages = interval_df.interval_min_ma[1:nrow(interval_df)-1]  
boundary_ages
```

```
69-element Vector{Float64}:
```

```
33.9  
31.8  
27.3  
27.3  
27.3  
27.3  
27.3  
23.04  
23.04  
23.04  
  
0.0  
0.774  
0.4  
0.21  
0.129  
0.0117  
0.0117  
0.014  
0.0
```

3.7.2 The diversity loop

The loop structure is the same as before. The only difference is the condition inside `count`: `>` instead of `>=` on both sides of the comparison.

```

diversity = Int[]

for b in boundary_ages
  n = count(
    g -> g.fad_ma > b > g.lad_ma,
    eachrow(genus_ranges)
  )
  push!(diversity, n)
end

```

3.7.3 Assembling the results table

We need a DataFrame that pairs each boundary age with its diversity count. We build one directly:

```

boundary_df = DataFrame(
  boundary_ma = boundary_ages,
  diversity    = diversity,
)

```

	boundary_ma	diversity
	Float64	Int64
1	33.9	1
2	31.8	0
3	27.3	5
4	27.3	5
5	27.3	5
6	27.3	5
7	27.3	5
8	23.04	5
9	23.04	5
10	23.04	5
11	23.04	5
12	18.5	3
13	18.5	3
14	20.45	3
15	20.0	3
16	18.5	3
17	16.9	6
18	15.98	6
19	16.3	6
20	16.3	6
21	11.63	6
22	15.98	6
23	12.5	6
24	12.75	6
...	...	...

`DataFrame(col1 = values1, col2 = values2, ...)` is a compact way to construct a `DataFrame` from scratch when you already have the column vectors.

We need to sort the age order on the x-axis:

```
boundary_df = sort(boundary_df, :boundary_ma, rev = true)
```


	boundary_ma	diversity
	Float64	Int64
1	33.9	1
2	31.8	0
3	27.3	5
4	27.3	5
5	27.3	5
6	27.3	5
7	27.3	5
8	23.04	5
9	23.04	5
10	23.04	5
11	23.04	5
12	20.45	3
13	20.0	3
14	18.5	3
15	18.5	3
16	18.5	3
17	16.9	6
18	16.3	6
19	16.3	6
20	15.98	6
21	15.98	6
22	12.75	6
23	12.5	6
24	11.63	6
...	...	...

3.8 Plotting the diversity curve

Same pattern as before — `lines!` plus `scatter!`, x-axis reversed. We use `boundary_ma` directly as the x-coordinate; no interval names to label because boundaries sit *between* intervals.

```
fig = Figure()
ax = Axis(
    fig[1, 1],
```

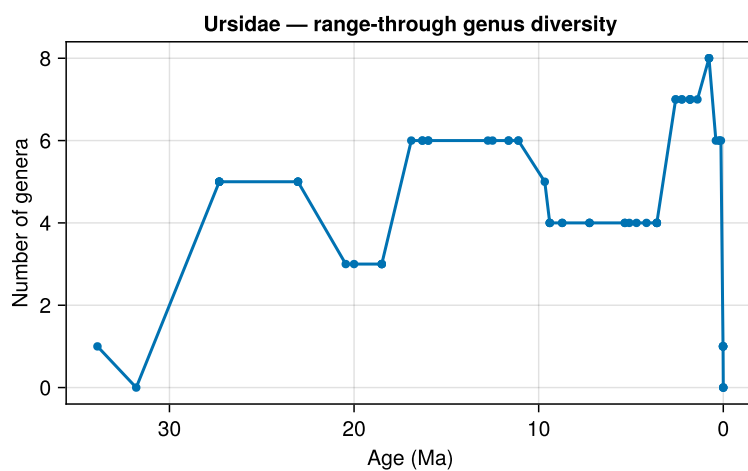
```

    xlabel = "Age (Ma)",
    ylabel = "Number of genera",
    title = "Ursidae - range-through genus diversity",
    xreversed = true,
)

lines!(ax, boundary_df.boundary_ma, boundary_df.diversity, linewidth = 2)
scatter!(ax, boundary_df.boundary_ma, boundary_df.diversity, markersize = 8)

fig

```



3.9 Complete program

For reference, here is the full program in one place:

```

using DataFrames
using PaleobiologyDB
using PaleobiologyDB.Taxonomy
using Makie, CairoMakie

set_autocaching!(true)

```

```

df = pbdb_occurrences(
    base_name = "Ursidae";
    show = "full",
    vocab = "pbdb"
)

df = dropmissing(df, [
    :genus,
    :early_interval,
    :max_ma,
    :min_ma
])
df = Taxonomy.drop_unqualified_taxa(df, "genus")

df.midpoint_age = (df.max_ma .+ df.min_ma) ./ 2

interval_df = combine(
    groupby(df, :early_interval),
    :max_ma => maximum => :interval_max_ma,
    :min_ma => minimum => :interval_min_ma,
)

interval_df.interval_mid_ma = (
    interval_df.interval_max_ma .+ interval_df.interval_min_ma
) ./ 2

interval_df = sort(interval_df, :interval_mid_ma, rev=true)

genus_ranges = combine(
    groupby(df, :genus),
    :midpoint_age => maximum => :fad_ma,
    :midpoint_age => minimum => :lad_ma,
)

boundary_ages = interval_df.interval_min_ma[1:nrow(interval_df)-1]
boundary_ages

diversity = Int[]

for b in boundary_ages

```

```

    n = count(
        g -> g.fad_ma > b > g.lad_ma,
        eachrow(genus_ranges)
    )
    push!(diversity, n)
end

boundary_df = DataFrame(
    boundary_ma = boundary_ages,
    diversity    = diversity,
)

boundary_df = sort(boundary_df, :boundary_ma, rev = true)

fig = Figure()
ax = Axis(
    fig[1, 1],
    xlabel = "Age (Ma)",
    ylabel = "Number of genera",
    title  = "Ursidae - range-through genus diversity",
    xreversed = true,
)

lines!(ax, boundary_df.boundary_ma, boundary_df.diversity, linewidth = 2)
scatter!(ax, boundary_df.boundary_ma, boundary_df.diversity, markersize = 8)

fig

```

3.10 Exercises: Evaluating richness through time — range-through method

3.10.1 Julia concepts and mechanisms

For each snippet, predict the output before running it in the REPL.

3.10.1.1 Part A: Strict vs. non-strict inequalities

1. The difference between `>` and `>=`:

```
println(5 > 5)
println(5 >= 5)
```

i Answer

false

true

`>` requires strictly greater; `>=` allows equality.

2. At a floating-point boundary:

```
x = 10.0
println(x > 10.0)
println(x >= 10.0)
```

i Answer

false

true

3. FAD equals boundary — range-through or not?

```
fad = 30.0
lad = 5.0
b   = 30.0
println(fad > b > lad)    # strict (range-through)
println(fad >= b >= lad)  # non-strict (sampled-in-bin)
```

i Answer

false

true

Strict: `30 > 30` is false — the genus was first seen *at* the boundary, not before it, so it cannot be confirmed on the older side. Non-strict: `30 >= 30` is true — the genus just barely qualifies.

4. LAD equals boundary:

```

fad = 15.0
lad = 8.0
b   = 8.0
println(fad > b > lad)    # strict
println(fad >= b >= lad) # non-strict

```

i Answer

false
true

Strict: $8 > 8$ is false — the genus disappeared *at* the boundary, not after it. Non-strict: $8 \geq 8$ is true.

5. How many values exceed a threshold with strict vs. non-strict?

```

data = [5.0, 10.0, 15.0, 20.0, 25.0]
b = 10.0
println(count(x -> x > b, data))
println(count(x -> x >= b, data))

```

i Answer

3
4

Strict > 10.0 : 15, 20, 25 (three values). Non-strict ≥ 10.0 : 10, 15, 20, 25 (four values).

6. Strict three-way comparison:

```

a, b, c = 10.0, 20.0, 10.0
println(a < b > c)

```

i Answer

true

$10 < 20$ is true; $20 > 10$ is true; both must hold, and they do.

7. Mixed strict and non-strict:

```
println(5 <= 5 < 10)
println(5 < 5 <= 10)
```

i Answer

```
true
false
```

First: 5 5 is true and 5 < 10 is true. Second: 5 < 5 is false — fails at the first step.

8. Comparing range-through and sampled-in-bin at the FAD boundary:

```
fad_ma = 25.0
lad_ma = 10.0
boundary = 25.0

range_through = fad_ma > boundary > lad_ma
sampled_in_bin = fad_ma >= boundary >= lad_ma

println("Range-through:  ", range_through)
println("Sampled-in-bin: ", sampled_in_bin)
```

i Answer

```
Range-through:  false
Sampled-in-bin:  true
```

The range-through method requires occurrences *before* the boundary; a genus whose FAD equals the boundary age has no confirmed record on the older side.

3.10.1.2 Part B: Array slicing

1. Extracting a slice:

```
v = [10, 20, 30, 40, 50]
println(v[1:3])
```

i Answer

[10, 20, 30]

1:3 includes indices 1, 2, and 3.

2. All but the last element — the boundary pattern:

```
v = [10, 20, 30, 40, 50]
n = length(v)
println(v[1:n-1])
```

i Answer

[10, 20, 30, 40]

1:n-1 stops one short of the end — exactly how we extract boundary ages from `interval_min_ma`.

3. A middle slice:

```
v = [10, 20, 30, 40, 50]
println(v[2:4])
```

i Answer

[20, 30, 40]

4. How many boundaries between `n` intervals?

```
intervals = [50.0, 40.0, 30.0, 20.0, 10.0]
boundaries = intervals[1:length(intervals)-1]
println(length(boundaries))
```

i Answer

4

Five intervals share four boundaries — one between each adjacent pair.

5. Extracting the younger edges as boundary ages:

```
interval_min_ma = [5.3, 2.6, 0.8, 0.0]
n = length(interval_min_ma)
```



```
boundary_ages = interval_min_ma[1:n-1]
println(boundary_ages)
```

i Answer

[5.3, 2.6, 0.8]

The last interval's younger edge (0.0 Ma, the present) is not a shared boundary with any younger interval, so it is excluded.

6. Computing n-1 explicitly:

```
n_intervals = 6
println(n_intervals - 1)
```

i Answer

5

7. Slice from index 3 to the end:

```
v = [1.0, 2.0, 3.0, 4.0, 5.0]
println(v[3:end])
```

i Answer

[3.0, 4.0, 5.0]

3:end includes every index from 3 onward.

8. Last two elements:

```
v = [1.0, 2.0, 3.0, 4.0, 5.0]
n = length(v)
println(v[n-1:n])
```

i Answer

[4.0, 5.0]

3.10.1.3 Part C: Range-through vs. sampled-in-bin logic

1. Sampled-in-bin counts a genus whose FAD equals the interval midpoint; range-through does not count it at the corresponding boundary:

```
fad, lad, mid = 20.0, 5.0, 20.0
sampled_in_bin = fad >= mid >= lad
range_through_rtm = fad > mid > lad
println(sampled_in_bin)
println(range_through_rtm)
```

i Answer

```
true
false
```

2. Range-through count at a boundary with four genera:

```
using DataFrames
genus_ranges = DataFrame(
    fad_ma = [30.0, 20.0, 15.0, 10.0],
    lad_ma = [10.0, 10.0, 5.0, 1.0],
)
b = 10.0
n = count(g -> g.fad_ma > b > g.lad_ma, eachrow(genus_ranges))
println(n)
```

i Answer

```
1
Row 1: 30 > 10 > 10? 10 > 10 is false → no. Row
2: 20 > 10 > 10? same → no. Row 3: 15 > 10 >
5? yes → yes. Row 4: 10 > 10? no → no.
```

3. Sampled-in-bin count for the same data and midpoint:

```
using DataFrames
genus_ranges = DataFrame(
    fad_ma = [30.0, 20.0, 15.0, 10.0],
    lad_ma = [10.0, 10.0, 5.0, 1.0],
)
mid = 10.0
n = count(g -> g.fad_ma >= mid >= g.lad_ma, eachrow(genus_ranges))
println(n)
```

i Answer

4

All four rows satisfy `fad_ma < 10` and `lad_ma` with non-strict inequalities. Compare to the range-through result above — the method difference is significant for genera whose ranges end exactly at a boundary.

4. How many boundaries does `interval_min_ma[1:n-1]` exclude?

```
interval_min_ma = [33.9, 23.0, 5.3, 2.6, 0.8, 0.0]
n = length(interval_min_ma)
boundary_ages = interval_min_ma[1:n-1]
println(length(interval_min_ma) - length(boundary_ages))
```

i Answer

1

One element (the last, representing the present) is excluded.

5. Building `boundary_df` from scratch:

```
using DataFrames
boundary_ages = [20.0, 15.0, 10.0, 5.0]
diversity      = [3, 5, 4, 2]
df = DataFrame(boundary_ma = boundary_ages, diversity = diversity)
println(nrow(df))
println(df.diversity[2])
```

i Answer

4

5

6. Sorting `boundary_df` from oldest to youngest:

```
using DataFrames
boundary_df = DataFrame(  

```

```

    boundary_ma = [5.0, 20.0, 10.0, 15.0],
    diversity    = [2,    3,    4,    5 ],
)
sorted = sort(boundary_df, :boundary_ma, rev=true)
println(sorted.boundary_ma)

```

i Answer

[20.0, 15.0, 10.0, 5.0]

rev=true places the largest (oldest) values first — the geological convention.

7. Number of boundaries from a five-row interval table:

```

using DataFrames
interval_df = DataFrame(
    name      = ["A", "B", "C", "D", "E"],
    max_ma    = [50.0, 40.0, 30.0, 20.0, 10.0],
    min_ma    = [40.0, 30.0, 20.0, 10.0, 0.0],
)
println(nrow(interval_df) - 1)

```

i Answer

4

Five intervals → four shared boundaries between them.

3.10.1.4 Part D: Comprehensive mini-pipelines

1. A complete range-through count over three boundaries:

```

using DataFrames
genus_ranges = DataFrame(
    fad_ma = [40.0, 20.0, 10.0],
    lad_ma = [10.0, 5.0, 2.0],
)
boundaries = [30.0, 15.0, 7.0]
diversity = Int[]
for b in boundaries

```

```

    n = count(g -> g.fad_ma > b > g.lad_ma, eachrow(genus_ranges))
    push!(diversity, n)
end
println(diversity)

```

i Answer

[1, 2, 2]

Boundary 30.0: only genus 1 (40>30>10) qualifies. Boundary 15.0: genera 1 (40>15>10) and 2 (20>15>5) qualify. Boundary 7.0: genera 2 (20>7>5) and 3 (10>7>2) qualify.

2. Length of the boundary vector:

```

interval_min_ma = [33.9, 23.0, 5.3, 2.6, 0.8]
boundary_ages = interval_min_ma[1:length(interval_min_ma)-1]
println(length(boundary_ages))

```

i Answer

4

Five intervals → four boundaries.

3. Sampled-in-bin counts more than range-through when FAD aligns with a boundary:

```

using DataFrames
fad_ma = [30.0, 25.0, 20.0, 15.0]
lad_ma = [ 5.0,  5.0,  5.0,  5.0]
boundary = 20.0

gr = DataFrame(fad_ma = fad_ma, lad_ma = lad_ma)
sib = count(g -> g.fad_ma >= boundary >= g.lad_ma, eachrow(gr))
rt = count(g -> g.fad_ma > boundary > g.lad_ma, eachrow(gr))
println("SIB: ", sib)
println("RT:  ", rt)

```

i Answer

SIB: 3

RT: 2

SIB counts genus 3 (FAD == boundary, satisfies >=);
RT does not (fails strict >).

4. Extracting and printing boundary ages:

```
v = [50.0, 40.0, 30.0, 20.0, 10.0, 0.0]
n = length(v)
boundaries = v[1:n-1]
println(boundaries)
println(length(boundaries))
```

i Answer

```
[50.0, 40.0, 30.0, 20.0, 10.0]
5
```

5. A genus whose range is a single point at a boundary:

```
fad = 10.0
lad = 10.0
b   = 10.0
println(fad > b > lad)    # strict
println(fad >= b >= lad) # non-strict
```

i Answer

false

true

Strict: 10 > 10 fails on both sides. Non-strict: 10
10 10 is true — but this is a degenerate case (a
point range).

6. Complete mini-pipeline with boundary ages from an interval table:

```
using DataFrames
genus_ranges = DataFrame(
    fad_ma = [50.0, 30.0, 15.0],
```

```

    lad_ma = [20.0, 5.0, 1.0],
  )
  interval_min_ma = [40.0, 25.0, 10.0, 2.0]
  n = length(interval_min_ma)
  boundary_ages = interval_min_ma[1:n-1]

  diversity = Int[]
  for b in boundary_ages
    push!(diversity, count(g -> g.fad_ma > b > g.lad_ma, eachrow(genus_ranges)))
  end
  println(diversity)

```

i Answer

[1, 2, 2]

Boundary 40.0: only genus 1 ($50 > 40 > 20$). Boundary 25.0: genera 1 ($50 > 25 > 20$) and 2 ($30 > 25 > 5$). Boundary 10.0: genera 2 ($30 > 10 > 5$) and 3 ($15 > 10 > 1$).

7. A genus with FAD and LAD at the same point cannot range through any boundary:

```

fad = 23.5
lad = 23.5
boundaries = [23.5, 20.0, 15.0]
for b in boundaries
  println(fad > b > lad)
end

```

i Answer

false

false

false

When `fad == lad`, the genus occupies a single age point and cannot bracket any boundary on both sides.

3.10.2 Practical shorts

Assume you have completed the tutorial and have `df`, `interval_df`, `genus_ranges`, `boundary_ages`, `boundary_df`, and the diversity curve available in your REPL.

1. How many boundaries are in `boundary_df`? Compare `nrow(boundary_df)` with `nrow(interval_df)`. What is the difference?
2. Which boundary age has the highest range-through diversity? `boundary_df$boundary_ma[argmax(boundary_df$diversity)]`
3. Which boundary has the lowest diversity? Is it 0? If so, what does that mean biologically?
4. Compute the mean range-through diversity: `sum(boundary_df$diversity) / nrow(boundary_df)`
5. Add a horizontal reference line at the mean diversity: `hlines!(ax, [mean_diversity], color = :gray, linestyle = :dash)`
6. Print all boundary ages where diversity drops compared to the preceding boundary. `diff(boundary_df$diversity)` gives the change between successive values; find where the result is negative.
7. Try changing `>` to `>=` in the boundary loop condition. For which boundaries, if any, does the count change?
8. Plot both the sampled-in-bin diversity curve (from the previous tutorial) and the range-through curve on the same axis in different colors. Add a legend.
9. For each boundary, print the names of the genera that are counted as present. Modify the loop to collect genus names rather than just a count.
10. Which genus appears at the most boundaries? Modify the counting loop to accumulate, for each genus, how many boundaries it spans.
11. What is the first boundary age (oldest) in `boundary_df` when sorted from oldest to youngest?

12. How does `nrow(interval_df) - 1` compare to `length(boundary_ages)`? They should be equal — verify it.
13. Try using `interval_df.interval_max_ma[2:end]` as boundary ages instead of `interval_df.interval_min_ma[1:n-1]`. Are the resulting boundary ages the same? Why or why not?
14. Export `boundary_df` to a CSV file: `CSV.write("ursidae_range_through.csv", boundary_df)`
15. Compute range-through diversity for Felidae and plot it alongside the Ursidae curve.
16. Plot the *difference* between sampled-in-bin and range-through diversity as its own curve. Where is the discrepancy largest?
17. Add larger, filled markers at the three boundaries with the highest diversity. Use a second `scatter!` call with a filtered subset of the data.
18. How many genera in Ursidae have a single-interval range (i.e., `fad_ma` and `lad_ma` fall within the same stratigraphic interval)? These genera cannot contribute to any range-through boundary count.
19. Rerun the range-through analysis using equal 5-Ma bins of your own construction instead of PBDB intervals. Does the shape of the curve change?
20. Create `boundary_df` *without* sorting and plot the diversity curve. What goes wrong visually, and why?
21. After sorting `boundary_df` by `boundary_ma` descending, print the `boundary_ma` column and confirm it runs from largest (oldest) to smallest (youngest).
22. Inspect the genera counted at the oldest boundary. Filter `genus_ranges` manually: which genera satisfy `fad_ma > oldest_boundary > lad_ma`?
23. Compute the maximum single-step drop in diversity between adjacent boundaries: `minimum(diff(boundary_df.diversity))`

24. Apply the range-through pipeline to a much older taxon, such as *Crocodylia*. Does the pipeline run without modification?
25. Change `markersize = 8` to `markersize = 14`, then try mapping marker size to the diversity value: `scatter!(ax, x, y, markersize = boundary_df.diversity .* 2)`
26. How many boundaries have diversity above 5? `count(x -> x > 5, boundary_df.diversity)`
27. Print the boundary ages where genus *Ursus* contributes to the count. *Ursus* contributes at boundary `b` when `fad_ma > b > lad_ma`; look up its FAD and LAD from `genus_ranges`.
28. Compare the range-through diversity at 10.0 Ma with the sampled-in-bin diversity at the interval whose midpoint is closest to 10.0 Ma. Do they match? If not, which is higher and why?
29. Modify the pipeline so that only genera with at least 3 fossil occurrences are counted toward range-through diversity. (Hint: first compute occurrence counts per genus and filter `genus_ranges` accordingly.)
30. Plot the range-through diversity, sampled-in-bin diversity, and the raw occurrence count per interval all on the same figure. Use a second y-axis for the occurrence count if the scales are very different.

3.10.3 Creative programming

3.10.3.1 Three-panel summary figure

Create a single figure summarising all three tutorials in one view: panel 1 shows the genus range chart (tutorial 1), panel 2 the sampled-in-bin diversity curve (tutorial 2), and panel 3 the range-through diversity curve (tutorial 3).

Objectives

- Arrange three panels in a single **Figure**: `fig[1,1]`, `fig[2,1]`, `fig[3,1]`.
- Use the same x-axis range and `xreversed = true` on all three panels.
- Add titles to each panel and a shared x-axis label at the bottom.

Hints

- `linkxaxes!(ax1, ax2, ax3)` keeps all three panels in sync when you zoom.
- Set `xlabel` only on the bottom panel to avoid repetition.

3.10.3.2 Range-through diversity for deep time

Apply the range-through pipeline to a group with a very long fossil record — for example Crocodilia (spanning the Triassic to the present) or Testudines (turtles).

Objectives

- Download and clean the data; note the number of intervals returned.
- Run the range-through pipeline without modification.
- Plot the diversity curve and identify the interval of peak diversity.
- Identify any sharp drops that might correspond to known extinction events.

Hints

- Very long-ranging taxa may span hundreds of intervals; the pipeline handles this without changes.
- Look up major extinction events (end-Cretaceous, end-Triassic) and check whether they are visible in the curve.

3.10.3.3 Origination and extinction rates

Use the boundary framework to compute, at each boundary, the number of genera appearing for the first time (originations) and the number disappearing (extinctions).

Objectives

- An origination at boundary `b` is a genus whose FAD falls in the interval immediately older than `b` (i.e., `older_interval_max >= fad_ma > b`).
- An extinction at `b` is a genus whose LAD falls in the interval immediately younger than `b`.
- Plot originations, extinctions, and diversity on the same figure with different colors.
- Identify the boundary with the highest extinction rate.

Hints

- You will need the interval max/min ages alongside the boundary ages; keep `interval_df` in scope.
- Use a single `Axis` with three lines! calls `axislegend`.

3.10.3.4 Monte Carlo uncertainty bands

The FAD and LAD estimates depend on which age you assign to each occurrence. Quantify this uncertainty.

Objectives

- For each occurrence, draw a random age uniformly from its `[min_ma, max_ma]` interval.
- Run the full range-through pipeline on this randomised dataset.
- Repeat 100 times; record the diversity at each boundary for each run.
- Plot the mean diversity curve with a shaded uncertainty band (the 10th–90th percentile).

Hints

- `rand() * (max - min) + min` generates a uniform random value between `min` and `max`.
- Collect the 100 diversity vectors into a matrix; compute percentiles column-by-column.
- `band!(ax, x, lower, upper)` draws a shaded region between two curves.

3.10.3.5 Lazarus taxa

A “Lazarus taxon” is a genus that apparently disappears from the fossil record for one or more intervals but then reappears later.

Objectives

- For each genus, count how many boundaries it spans (FAD > boundary > LAD).
- Compare this to the number of intervals in which it actually has recorded occurrences.
- Flag genera where the span-to-occurrence ratio is high (spans 5+ boundaries but has occurrences in fewer than half the corresponding intervals).
- List the flagged genera and their apparent gap durations.

Hints

- The “span” comes from the range-through logic; the “actual occurrences” require going back to the raw `df` and counting per genus per interval.

3.10.3.6 Global vs. regional range-through diversity

Compare range-through diversity curves for a taxon of your choice restricted to different continents.

Objectives

- Filter the cleaned occurrence data to at least two continental regions.
- Run the range-through pipeline independently for each region using the *same* set of intervals.
- Plot the regional curves on the same figure with different colors.
- Describe one pattern that differs between the regions.

Hints

- Use the `cc` field (continent code) in the occurrence data. Filter with `df[df.cc != "NOA", :]` for North America (verify the exact code in your data).

- Keeping the same `interval_df` for both regions ensures the time axes are comparable.

3.10.3.7 Publication-ready diversity figure

Take the range-through diversity curve and upgrade it to a figure suitable for a journal article.

Objectives

- Apply a clean, professional color scheme.
- Add labeled vertical lines at major Cenozoic epoch boundaries (Eocene/Oligocene 33.9 Ma, Oligocene/Miocene 23.0 Ma, Miocene/Pliocene 5.3 Ma, Pliocene/Pleistocene 2.6 Ma).
- Use clean axis labels with units.
- Export as PDF or SVG with `save("figure.pdf", fig)`.

Hints

- Epoch boundary lines: `vlines!(ax, [33.9, 23.0, 5.3, 2.6], color = :lightgray)`.
- `text!(ax, x, ylims[2], text = "Miocene", fontsize = 9, align = (:center, :top))` places epoch labels at the top of the axes.

3.10.3.8 Comparing methods across five families

For five mammal families of your choice, compute both the sampled-in-bin and range-through diversity at their peak and compare the two methods systematically.

Objectives

- For each family, run both pipelines and record the peak diversity and the ratio `range_through_peak / sampled_in_bin_peak`.
- Create a scatter plot: sampled-in-bin peak on the x-axis, range-through peak on the y-axis.
- Add a diagonal line ($y = x$) as a reference.

Hints

- Store results in a `DataFrame` with columns `:family`, `:sib_peak`, `:rt_peak`.
- The diagonal reference line: `lines!(ax, [0, max_val], [0, max_val], linestyle = :dash)`.

3.10.3.9 Custom geological time scale

Replace the PBDB interval boundaries with the formal geological stage boundaries from the International Chronostratigraphic Chart (ICS).

Objectives

- Research and hard-code the boundary ages for the Neogene and Quaternary stages (e.g., Gelasian 2.588 Ma, Zanclean 5.333 Ma, Messinian 7.246 Ma, ...).
- Build a custom `boundary_ages` vector from these values.
- Run the range-through diversity loop with these boundaries.
- Compare the curve shape with the PBDB-interval version.

Hints

- The ICS chart is available at stratigraphy.org.
- Hard-code the boundary vector in descending order (oldest first) to match the pipeline's expected ordering.

3.10.3.10 Interactive slider for minimum occurrence count

Using the GLMakie backend (interactive), add a slider that controls the minimum number of fossil occurrences required for a genus to be included in the range-through count.

Objectives

- Install GLMakie (`] add GLMakie`) and replace `using CairoMakie` with `using GLMakie`.
- Add an `Observable` and a `Slider` widget that controls a minimum-occurrences threshold.
- Recompute `genus_ranges` whenever the slider value changes, filtering out genera below the threshold.

- Watch the diversity curve update in real time as you drag the slider.

Hints

- `GLMakie.Slider` and `Observable` are the key types; search the Makie documentation for “slider observable”.
- Start with a threshold range of 1 to 10 occurrences.

3.11 Appendix: Split-Apply-Combine golf

The split-apply-combine pattern — `combine(groupby(df, key), transforms...)` — is the workhorse of `DataFrame` analysis in Julia. This appendix gives you two sets of exercises to sharpen your fluency with it.

The first set asks you to *predict* what a given `combine` expression produces. The second set asks you to *write* the most concise expression that produces a specified result — coding golf. All snippets are self-contained; copy them directly into the REPL.

3.11.1 Predict the result

For each snippet, work out the output before running it. Every example uses only `DataFrames` and base Julia — no `PBDB` data required.

1. One summary column:

```
using DataFrames
df = DataFrame(
    group = ["A", "A", "B", "B", "B"],
    val    = [1, 3, 2, 4, 6],
)
result = combine(groupby(df, :group), :val => sum => :total)
println(result)
```


i Answer

2×2 DataFrame

Row	group	total
	String	Int64

1	A	4
2	B	12

Group A: 1+3=4. Group B: 2+4+6=12.

2. Two summary columns in one call:

```
using DataFrames
df = DataFrame(
    genus = ["X", "X", "Y", "Y", "Y"],
    age   = [10.0, 40.0, 5.0, 15.0, 20.0],
)
result = combine(
    groupby(df, :genus),
    :age => maximum => :fad,
    :age => minimum => :lad,
)
println(result.fad)
println(result.lad)
```

i Answer

[40.0, 20.0]

[10.0, 5.0]

X: max=40, min=10. Y: max=20, min=5.

3. Counting rows per group with `length`:

```
using DataFrames
df = DataFrame(
    family = ["Bear", "Bear", "Bear", "Cat", "Cat"],
    genus  = ["A", "B", "A", "C", "D"],
)
result = combine(groupby(df, :family), :genus => length => :n)
println(result.n)
```

i Answer

[3, 2]

`length` counts the number of elements in the group's column vector — equivalent to counting rows in the group.

4. How many rows does `combine` produce?

```
using DataFrames
df = DataFrame(
  x = ["a", "a", "b", "c", "c", "c", "d"],
  y = 1:7,
)
result = combine(groupby(df, :x), :y => sum => :s)
println(nrow(result))
```

i Answer

4

There are four distinct values in `:x` (a, b, c, d) → four rows in `result`.

5. Applying a custom function:

```
using DataFrames
df = DataFrame(
  group = ["A", "A", "B", "B"],
  val    = [3.0, 7.0, 1.0, 5.0],
)
result = combine(
  groupby(df, :group),
  :val => (v -> maximum(v) - minimum(v)) => :range,
)
println(result.range)
```

i Answer

[4.0, 4.0]

A: 7-3=4. B: 5-1=4. The anonymous function `v -> maximum(v) - minimum(v)` computes the range of

each group's values.

6. `mean` as the aggregation function:

```
using DataFrames, Statistics
df = DataFrame(
    site = ["N","N","S","S","S"],
    temp = [10.0, 14.0, 20.0, 22.0, 24.0],
)
result = combine(groupby(df, :site), :temp => mean => :mean_temp)
println(result.mean_temp)
```

i Answer

[12.0, 22.0]

N: $(10+14)/2 = 12$. S: $(20+22+24)/3 = 22$.

7. Counting unique values per group:

```
using DataFrames
df = DataFrame(
    region = ["E","E","E","W","W"],
    taxon = ["Bear","Bear","Cat","Dog","Bear"],
)
result = combine(
    groupby(df, :region),
    :taxon => (t -> length(unique(t))) => :n_taxa,
)
println(result.n_taxa)
```

i Answer

[2, 2]

East: unique taxa are Bear and Cat $\rightarrow 2$. West:
Bear and Dog $\rightarrow 2$.

8. Chaining `sort` after `combine`:

```
using DataFrames
df = DataFrame(
    sp = ["B","A","A","C","B","C","C"],
    count = [1, 2, 3, 4, 5, 6, 7],
)
```

```

)
result = combine(groupby(df, :sp), :count => sum => :total)
result = sort(result, :total, rev=true)
println(result.sp)

```

i Answer

["C", "B", "A"]

Totals: A=5, B=6, C=17. Sorted descending: C, B, A.

9. `nrow` as the aggregation function:

```

using DataFrames
df = DataFrame(
    stage = ["Mio", "Mio", "Plio", "Plio", "Plio", "Plei"],
    taxon  = ["A", "B", "A", "C", "D", "A"],
)
result = combine(groupby(df, :stage), nrow => :count)
println(result.count)

```

i Answer

[2, 3, 1]

`nrow` (without a column specification) counts rows directly in each group. Miocene: 2 rows. Pliocene: 3. Pleistocene: 1.

10. Multiple groups — two-column `groupby`:

```

using DataFrames
df = DataFrame(
    continent = ["EU", "EU", "NA", "NA", "NA"],
    family     = ["Bear", "Cat", "Bear", "Bear", "Cat"],
    n          = [3, 1, 5, 2, 4],
)
result = combine(
    groupby(df, [:continent, :family]),
    :n => sum => :total,
)
println(nrow(result))

```

```
println(result.total)
```

i Answer

3

[3, 1, 7, 4]

Three unique (continent, family) pairs: (EU, Bear)→3, (EU, Cat)→1, (NA, Bear)→5+2=7, (NA, Cat)→4.

11. The result column name when => is omitted:

```
using DataFrames
df = DataFrame(g = ["A","A","B"], v = [1,2,3])
result = combine(groupby(df, :g), :v => sum)
println(names(result))
```

i Answer

["g", "v_sum"]

When you write `:v => sum` without the third `=>`, DataFrames automatically names the result column after the source column and the function: `v_sum`.

12. `first` and `last` as aggregation functions:

```
using DataFrames
df = DataFrame(
    grp = ["A","A","A","B","B"],
    val = [10, 20, 30, 40, 50],
)
result = combine(
    groupby(df, :grp),
    :val => first => :earliest,
    :val => last  => :latest,
)
println(result.earliest)
println(result.latest)
```

i Answer

[10, 40]

[30, 50]

`first` takes the first element of the group's column;
`last` takes the last.

13. Aggregation producing a Boolean column:

```
using DataFrames
df = DataFrame(
    genus = ["A","A","B","B","B"],
    extant = [true, false, true, true, true],
)
result = combine(
    groupby(df, :genus),
    :extant => any => :any_extant,
)
println(result.any_extant)
```

i Answer

[true, true]

`any` returns `true` if at least one element in the group is `true`. Both genera A and B have at least one `true` record.

14. Passing the whole sub-DataFrame to a function:

```
using DataFrames
df = DataFrame(
    group = ["A","A","B","B"],
    x      = [1.0, 3.0, 2.0, 6.0],
    y      = [4.0, 2.0, 5.0, 1.0],
)
result = combine(
    groupby(df, :group),
    [:x, :y] => ((x, y) -> sum(x) / sum(y)) => :ratio,
)
println(result.ratio)
```

i Answer

[0.6666666666666666, 1.0]

A: $(1+3)/(4+2) = 4/6 = 0.667$. B: $(2+6)/(5+1) = 8/8 = 1.0$. Passing multiple source columns uses the `[:col1, :col2] => function` syntax.

15. What happens when a group has only one row?

```
using DataFrames
df = DataFrame(
    genus = ["A", "B", "B", "C"],
    age   = [10.0, 20.0, 30.0, 15.0],
)
result = combine(
    groupby(df, :genus),
    :age => maximum => :fad,
    :age => minimum => :lad,
)
println(result[result.genus .== "A", [:fad, :lad]])
```

i Answer

```
1×2 DataFrame
 Row  fad      lad
      Float64  Float64

  1    10.0    10.0
```

When a group has only one row, `maximum` and `minimum` both return that single value — FAD equals LAD, meaning a point range.

16. Sorting within `groupby` using `sort = true`:

```
using DataFrames
df = DataFrame(genus = ["B", "A", "A", "B"], age = [5, 10, 20, 15])
r1 = combine(groupby(df, :genus; sort=true), :age => sum => :s)
r2 = combine(groupby(df, :genus; sort=false), :age => sum => :s)
println(r1.genus)
println(r2.genus)
```

i Answer

```
["A", "B"]  
["B", "A"]
```

`sort=true` orders groups alphabetically.
`sort=false` returns groups in the order they are first encountered in `df`.

17. The identity function as a passthrough:

```
using DataFrames  
df = DataFrame(g = ["X","X","Y"], v = [1,2,3])  
result = combine(groupby(df, :g), :v => identity => :v_copy)  
println(result.v_copy)
```

i Answer

```
[1, 2, 3]
```

`identity` returns its argument unchanged. `combine` with `identity` on a column returns all values flattened back into a single column — the group structure is expanded rather than summarised.

18. How many output rows when using `identity`?

```
using DataFrames  
df = DataFrame(g = ["A","A","B","B","B"], v = 1:5)  
result_sum = combine(groupby(df, :g), :v => sum)  
result_iden = combine(groupby(df, :g), :v => identity)  
println(nrow(result_sum))  
println(nrow(result_iden))
```

i Answer

```
2  
5
```

`sum` produces one scalar per group → 2 rows.
`identity` keeps all values → 5 rows (same as the original `DataFrame`).

19. Aggregating a string column:


```
using DataFrames
df = DataFrame(
    family = ["Bear", "Bear", "Cat"],
    genus  = ["Ursus", "Ailuropoda", "Felis"],
)
result = combine(
    groupby(df, :family),
    :genus => (g -> join(g, ", ")) => :genera_list,
)
println(result.genera_list)
```

i Answer

```
["Ursus", "Ailuropoda", "Felis"]
```

The anonymous function `g -> join(g, ", ")` concatenates the genus names in each group into a single comma-separated string.

20. Nested combine calls — summarising the summary:

```
using DataFrames
df = DataFrame(
    continent = ["EU", "EU", "NA", "NA", "EU"],
    family     = ["Bear", "Cat", "Bear", "Dog", "Bear"],
)
# Step 1: count taxa per continent
step1 = combine(groupby(df, :continent), nrow => :n_taxa)
# Step 2: find the continent with the most taxa
println(step1.continent[argmax(step1.n_taxa)])
```

i Answer

```
EU
```

EU has 3 rows; NA has 2. `argmax` returns the index of 3, which corresponds to EU.

3.11.2 Coding golf

Each challenge gives you a DataFrame and a target output. Write the shortest correct Julia expression that produces that output using `combine(groupby(...), ...)`. A model solution is in the answer — but there is often more than one way.

1. Sum per group.

```
using DataFrames
df = DataFrame(g = ["A","A","B","B"], v = [1,2,3,4])
# Target: DataFrame with columns :g and :total
# where :total is the sum of :v per group.
```

i Model solution

```
combine(groupby(df, :g), :v => sum => :total)
```

2. Row count per group.

```
using DataFrames
df = DataFrame(family = ["Bear","Bear","Cat","Cat","Cat"])
# Target: DataFrame with columns :family and :n,
# where :n is the number of rows per family.
```

i Model solution

```
combine(groupby(df, :family), nrow => :n)
```

3. Max and min in one call.

```
using DataFrames
df = DataFrame(genus = ["X","X","Y"], age = [5.0, 20.0, 8.0])
# Target: one row per genus with columns :fage (max age) and :lage (min age).
```

i Model solution

```
combine(groupby(df, :genus), :age => maximum => :fage, :age => minimum => :lage)
```

4. Range (max — min) per group.

```
using DataFrames
df = DataFrame(g = ["A","A","B","B"], v = [1.0, 5.0, 3.0, 3.5])
# Target: one row per group with column :span = max(v) - min(v).
```

i Model solution

```
combine(groupby(df, :g), :v => (v -> maximum(v) - minimum(v)) => :span)
```

5. Mean per group.

```
using DataFrames, Statistics
df = DataFrame(site = ["N","N","S","S"], val = [4.0, 8.0, 1.0, 3.0])
# Target: one row per site with column :avg = mean of val.
```

i Model solution

```
combine(groupby(df, :site), :val => mean => :avg)
```

6. Count unique values per group.

```
using DataFrames
df = DataFrame(
    region = ["E","E","W","W","W"],
    taxon  = ["Bear","Cat","Bear","Bear","Dog"],
)
# Target: one row per region with column :n_unique = number of distinct taxa.
```

i Model solution

```
combine(groupby(df, :region), :taxon => (t -> length(unique(t))) => :n_unique)
```

7. Sorted summary table.

```
using DataFrames
df = DataFrame(g = ["B","A","B","A","C"], v = [1,2,3,4,5])
# Target: sum of v per group, sorted by sum descending.
```

i Model solution

```
sort(combine(groupby(df, :g), :v => sum => :s), :s, rev=true)
```

8. Group with the largest sum.

```
using DataFrames
df = DataFrame(g = ["A","A","B","B","B"], v = [1,2,10,3,4])
# Target: a single String - the name of the group whose :v sum is largest.
```

i Model solution

```
let r = combine(groupby(df, :g), :v => sum => :s)
    r.g[argmax(r.s)]
end
```

The `let` block keeps the intermediate result out of the global namespace.

9. Proportion each group contributes to the total.

```
using DataFrames
df = DataFrame(g = ["A","A","B","B","B"], v = [1,2,3,4,5])
# Target: one row per group with column :pct = group sum / overall total * 100.
```

i Model solution

```
let total = sum(df.v)
    combine(groupby(df, :g), :v => (v -> 100 * sum(v) / total) => :pct)
end
```

The `total` is captured from the outer scope inside the anonymous function.

10. Boolean: does any row in the group satisfy a condition?

```
using DataFrames
df = DataFrame(
    genus = ["A","A","B","B"],
    extant = [true, false, false, false],
)
# Target: one row per genus with column :any_extant (Bool).
```

i Model solution

```
combine(groupby(df, :genus), :extant => any => :any_extant)
```

11. **Boolean: do all rows in the group satisfy a condition?**

```
using DataFrames
df = DataFrame(
  genus = ["A","A","B","B"],
  extant = [true, true, true, false],
)
# Target: one row per genus with column :all_extant (Bool).
```

i Model solution

```
combine(groupby(df, :genus), :extant => all => :all_extant)
```

12. **Concatenate strings per group.**

```
using DataFrames
df = DataFrame(
  family = ["Bear","Bear","Cat"],
  genus = ["Ursus","Ailuropoda","Felis"],
)
# Target: one row per family with column :genera = comma-joined genus names.
```

i Model solution

```
combine(groupby(df, :family), :genus => (g -> join(g, ", ")) => :genera)
```

13. **Add group-level statistics back to the original rows.**

```
using DataFrames
df = DataFrame(g = ["A","A","B","B","B"], v = [1,2,3,4,5])
# Target: the original df with an extra column :group_mean containing
# the mean of v for that row's group.
# Hint: use transform instead of combine.
```

i Model solution

```
transform(groupby(df, :g), :v => mean => :group_mean)
```

`transform` is like `combine` but keeps all original rows, broadcasting the group-level result back to every row in the group.

14. Rank rows within each group.

using DataFrames

```
df = DataFrame(g = ["A","A","B","B"], v = [10, 30, 5, 20])  
# Target: the original df with a column :rank = position of that row  
# when the group is sorted by v ascending (1 = smallest).
```

i Model solution

```
transform(groupby(df, :g), :v => (v -> sortperm(sortperm(v))) => :rank)
```

`sortperm(v)` gives the indices that would sort `v`; applying `sortperm` again gives the rank of each element.

15. Normalise values within each group.

using DataFrames

```
df = DataFrame(g = ["A","A","B","B","B"], v = [2.0, 4.0, 1.0, 3.0, 5.0])  
# Target: a column :v_norm = (v - group_min) / (group_max - group_min)  
# computed within each group.
```

i Model solution

```
transform(  
  groupby(df, :g),  
  :v => (v -> (v .- minimum(v)) ./ (maximum(v) - minimum(v))) => :v_norm,  
)
```

16. Count rows matching a condition per group.

using DataFrames

```
df = DataFrame(  
  family = ["Bear","Bear","Bear","Cat","Cat"],  
  age_ma = [1.0, 5.0, 50.0, 0.5, 10.0],
```

```
)
# Target: one row per family with column :n_old = count of records with age_ma > 5.
```

i Model solution

```
combine(groupby(df, :family), :age_ma => (a -> count(x -> x > 5, a)) => :n_old)
```

17. **Find the row with the maximum value per group.**

```
using DataFrames
df = DataFrame(
  group = ["A","A","B","B"],
  label = ["p","q","r","s"],
  score = [3, 7, 2, 9],
)
# Target: one row per group containing the label of the highest-scoring row.
```

i Model solution

```
combine(groupby(df, :group), [:label, :score] => ((l, s) -> l[argmax(s)]) => :best)
```

18. **Pivot: count occurrences of each value across groups.**

```
using DataFrames
df = DataFrame(
  continent = ["EU","EU","NA","NA","NA","AS"],
  status     = ["extant","extinct","extant","extant","extinct","extant"],
)
# Target: one row per continent with columns :n_extant and :n_extinct.
```

i Model solution

```
combine(
  groupby(df, :continent),
  :status => (s -> count(=="extant", s)) => :n_extant,
  :status => (s -> count(=="extinct", s)) => :n_extinct,
)
=="extant") is a partially applied equality function
```

— equivalent to `x -> x == "extant"`.

19. **Running total (cumulative sum) within each group.**

```
using DataFrames
df = DataFrame(g = ["A","A","A","B","B"], v = [1,2,3,10,20])
# Target: the original df with a column :cumsum_v = cumulative sum of v
# restarted for each group.
```

i Model solution

```
transform(groupby(df, :g), :v => cumsum => :cumsum_v)
```

20. **Two-column groupby: count per (continent, family) pair.**

```
using DataFrames
df = DataFrame(
  continent = ["EU","EU","NA","NA","EU"],
  family    = ["Bear","Cat","Bear","Dog","Bear"],
)
# Target: one row per unique (continent, family) pair with column :n = row count.
```

i Model solution

```
combine(groupby(df, [:continent, :family]), nrow => :n)
```